UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

JOÃO VICTOR DOS SANTOS GOMES

META-APRENDIZADO PARA AVALIAÇÃO DA QUALIDADE DO PRÉ-PROCESSAMENTO E RECOMENDAÇÃO OTIMIZADA DE PIPELINES EM AUTOML

MEDIANEIRA

2026

JOÃO VICTOR DOS SANTOS GOMES

META-APRENDIZADO PARA AVALIAÇÃO DA QUALIDADE DO PRÉ-PROCESSAMENTO E RECOMENDAÇÃO OTIMIZADA DE PIPELINES EM AUTOML

Meta-learning for Preprocessing Quality Assessment and Optimized Pipeline Recommendation in AutoML

Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Ciência da Computação do Curso de Bacharelado em Ciência da Computação da Universidade Tecnológica Federal do Paraná.

Orientador: Prof. Dr. Jorge Aikes Junior

Coorientador: Prof. Dr. Davi Pereira dos Santos

MEDIANEIRA

2026

4.0 Internacional

Esta licença permite remixe, adaptação e criação a partir do trabalho, para fins não comerciais, desde que sejam atribuídos créditos ao(s) autor(es) e que licenciem as novas criações sob termos idênticos. Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

JOÃO VICTOR DOS SANTOS GOMES

META-APRENDIZADO PARA AVALIAÇÃO DA QUALIDADE DO PRÉ-PROCESSAMENTO E RECOMENDAÇÃO OTIMIZADA DE PIPELINES EM AUTOML

Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Ciência da Computação do Curso de Bacharelado em Ciência da Computação da Universidade Tecnológica Federal do Paraná.

Prof. Dr. Jorge Aikes Junior
Orientador
Universidade Tecnológica Federal do Paraná

Prof. Dr. Davi Pereira dos Santos
Coorientador
Universidade Tecnológica Federal do Paraná

Prof. Dr. Jandrei Sartori Spancerski
Membro da banca examinadora
Universidade Tecnológica Federal do Paraná

MEDIANEIRA

2026

AGRADECIMENTOS

Agradeço ao meu orientador, Prof. Dr. Jorge Aikes Junior, e ao meu coorientador, Prof. Dr. Davi Pereira dos Santos, pela dedicação, pela disponibilidade e pelos direcionamentos que tornaram possível a realização deste trabalho.

Agradeço aos meus pais, pelo apoio incondicional e pelo incentivo constante ao longo de toda a minha trajetória acadêmica.

Agradeço também aos amigos que estiveram comigo ao longo dos anos de faculdade, compartilhando as dificuldades e as conquistas dessa caminhada.

Por fim, agradeço à Universidade Tecnológica Federal do Paraná (UTFPR), Câmpus Medianeira, pela estrutura e pelo apoio institucional oferecidos durante o desenvolvimento deste trabalho.

RESUMO

A seleção automática de pipelines de aprendizado de máquina é um problema central em AutoML, pois o desempenho dos modelos depende das etapas de pré-processamento e da combinação de algoritmos empregados. Este trabalho propõe uma abordagem de meta-aprendizado para recomendação e ranqueamento de pipelines de classificação baseada em uma representação estatística distribucional de conjuntos de dados, sem treinamento neural. Na Fase 1, construiu-se um meta-conjunto com 201 conjuntos obtidos do OpenML e 60 pipelines formados por combinações de escalonadores, seletores de atributos e classificadores, avaliados por validação cruzada estratificada. Cada conjunto foi representado por um vetor estatístico de 64 dimensões, inspirado nos critérios de representação do Dataset2Vec. Na Fase 2, um preditor k-NN multi-saída com estrutura KDTree foi avaliado por Leave-One-Out (LOO), utilizando Spearman e Kendall Tau para correlação global do ranking, e Kendall Tau ponderado e NDCG como métricas complementares voltadas ao topo da recomendação. Para $k = 10$, o preditor obteve Spearman médio de $0{,}58 \pm 0{,}36$, Kendall de $0{,}48 \pm 0{,}34$ e Kendall ponderado de $0{,}52 \pm 0{,}35$, indicando capacidade de capturar a ordem relativa dos pipelines, sobretudo nas primeiras posições. Uma demonstração com conjunto de dados retido é apresentada apenas como exemplo operacional do recomendador, sem substituir a validação estatística por LOO. A correlação entre dimensões da representação e Kendall ponderado ($p < 0{,}0001$) sugere que o vetor estatístico captura estrutura relevante para a recomendação. Os resultados indicam que representações estatísticas aliadas a k-NN constituem uma abordagem viável e reprodutível para recomendação de pipelines em AutoML.

Palavras-chave: meta-aprendizado; automl; recomendação de pipelines; ranqueamento; embeddings de datasets.

ABSTRACT

The automatic selection of machine learning pipelines is a central problem in AutoML, since model performance depends on preprocessing choices and algorithm combinations. This work proposes a meta-learning approach for recommending and ranking classification pipelines based on a distributional statistical representation of datasets, without neural training. In Phase 1, a meta-dataset was built from 201 OpenML datasets and 60 pipelines composed of scalers, feature selectors, and classifiers, evaluated with stratified cross-validation. Each dataset was represented by a 64-dimensional statistical vector inspired by the representation criteria of Dataset2Vec. In Phase 2, a multi-output k-nearest neighbors predictor with KDTree structure was evaluated through Leave-One-Out (LOO), using Spearman and Kendall Tau for global ranking correlation, and weighted Kendall Tau and NDCG as complementary top-ranking metrics. For $k = 10$, the predictor achieved a mean Spearman correlation of 0.58 ± 0.36, Kendall of 0.48 ± 0.34, and weighted Kendall of 0.52 ± 0.35, indicating its ability to capture the relative ordering of pipelines, especially at the top positions. A held-out dataset demonstration is reported only as an operational example of the recommender and does not replace the statistical LOO validation. The correlation between representation dimensions and weighted Kendall ($p < 0.0001$) suggests that the statistical vector captures structure relevant to recommendation. The results indicate that statistical representations combined with k-NN provide a viable and reproducible approach to AutoML pipeline recommendation.

Keywords: meta-learning; automl; pipeline recommendation; ranking; dataset embeddings.

LISTA DE FIGURAS

Figura 1 – Fluxograma do processo experimental: Fase 1 - Construção do meta-conjunto **29**

Figura 2 – Fluxograma do processo experimental: Fase 2 - Meta-aprendizado e recomendação no processo experimental **30**

Figura 3 – Distribuição de acurácias balanceadas na matriz $\mathbf{Y}_{acc}$ do meta-conjunto ($n = 12.060$) **38**

Figura 4 – Distribuição de acurácias balanceadas por classificador (todos os 201 conjuntos de dados) **39**

Figura 5 – Mapa de calor da acurácia balanceada média por escalador e classificador **39**

Figura 6 – Mapa de calor da acurácia balanceada média por seletor de atributos e classificador **40**

Figura 7 – Projeção UMAP dos embeddings de 201 conjuntos de dados no espaço bidimensional **41**

Figura 8 – Matriz de similaridade cosseno entre os embeddings dos 201 conjuntos de dados **42**

Figura 9 – Spearman e Kendall Tau em função de k (LOO sobre 201 conjuntos de dados) **44**

Figura 10 – Distribuição de Spearman e Kendall Tau por fold LOO ($k = 10$, 198 folds) **45**

Figura 11 – Distribuição do Kendall Tau ponderado por conjunto de dados e comparação com τ padrão **46**

LISTA DE TABELAS

Tabela 2 – Comparativo entre trabalhos relacionados e a proposta deste TCC . . . **25**

Tabela 3 – Exemplos de conjuntos de dados utilizados no meta-conjunto **28**

Tabela 4 – Arsenal de 60 pipelines: componentes disponíveis por estágio **36**

Tabela 5 – Resultados LOO do preditor k-NN para ranqueamento de pipelines (201 conjuntos de dados) **43**

Tabela 6 – Top-5 dimensões do embedding correlacionadas com o Kendall Tau ponderado por conjunto de dados ($n = 198$, LOO $k = 10$) **47**

Tabela 7 – Métricas de ranking para a demonstração ilustrativa com conjunto de dados retido (`task_id = 363713`, $k = 10$) **48**

Tabela 8 – Comparação do preditor k-NN ($k = 10$) com baselines triviais (LOO, $n = 198$ folds) **49**

Tabela 9 – Posicionamento dos resultados em relação à literatura de meta-aprendizado **49**

LISTA DE QUADROS

Quadro 1 – Materiais, Ferramentas e Conjuntos de Dados Utilizados 26

LISTA DE ABREVIATURAS E SIGLAS

Siglas

ANOVA	*Analysis of Variance*
AutoML	*Automated Machine Learning*
BADGE	*Batch Active learning by Diverse Gradient Embeddings*
BCE	*Binary Cross-Entropy*
CIFAR-10	*Canadian Institute for Advanced Research 10 classes*
CIFAR-100	*Canadian Institute for Advanced Research 100 classes*
D3M	*Data-Driven Discovery of Models*
DAG	*Directed Acyclic Graph*
DARPA	*Defense Advanced Research Projects Agency*
DBTC	*Decision Boundary Topological Complexity*
DT	*Decision Tree*
FashionMNIST	*Fashion Modified National Institute of Standards and Technology database*
ISIC2019	*International Skin Imaging Collaboration 2019 Dataset*
k-NN	*k-Nearest Neighbors*
LOO	*Leave-One-Out*
LSTM	*Long Short-Term Memory*
MEGA	*Meta-learning Guided Algorithm Recommendation*
ML	*Machine Learning*
MLP	*Multilayer Perceptron*
MNIST	*Modified National Institute of Standards and Technology database*
NDCG	*Normalized Discounted Cumulative Gain*
PCA	*Principal Component Analysis*
RF	*Random Forest*
SVM	*Support Vector Machine*
TCM	*TypiClust-Margin*
UCI	*University of California, Irvine*

SUMÁRIO

1 Introdução 12

1.1 Objetivos 13

1.2 Justificativa 13

1.3 Estrutura do Trabalho 14

2 Referencial Teórico 15

2.1 Aprendizado de Máquina e AutoML 15

2.2 Meta-aprendizado 16

2.3 Meta-características e Caracterização de Conjuntos de Dados 17

2.4 Aprendizado Ativo 17

2.5 Seleção e estrutura de *pipelines* 18

2.6 Componentes do Arsenal Experimental 19

2.7 Embeddings de Conjuntos de Dados e Métricas de Ranking 20

2.7.1 Embeddings de Conjuntos de Dados 20

2.7.2 Métricas de Avaliação de Ranking 21

2.8 Trabalhos Relacionados 22

2.8.1 Representação de Conjuntos de Dados e Meta-características 22

2.8.2 Sistemas de Recomendação de Algoritmos e Pipelines 23

2.8.3 Avaliação de Fronteiras de Decisão e Aprendizado Ativo 24

2.8.4 Síntese Comparativa 25

3 Materiais e Métodos 26

3.1 Materiais Utilizados 26

3.1.1 Linguagens e Ambientes de Desenvolvimento 27

3.1.2 Bibliotecas e Frameworks 27

3.1.3 Conjuntos de Dados 27

3.1.4 Recursos Computacionais 28

3.2 Métodos 29

3.2.1 Fase 1 – Construção do Meta-conjunto 30

3.2.2 Fase 2 – Meta-aprendizado por Ranqueamento 32

3.2.2.1 Preditor k-NN Multi-saída 32

3.2.2.2 Validação por Leave-One-Out 32

3.2.2.3 Métricas de Avaliação de Ranking . . . 33
3.2.2.4 Análise de Correlação Estatística . . . 34
4 Experimentos e Resultados . . . 35
4.1 Configuração Experimental . . . 35
4.1.1 Arsenal de Pipelines . . . 35
4.1.2 Fonte de Dados . . . 36
4.1.3 Protocolo de Validação . . . 36
4.2 Resultados da Fase 1 – Construção do Meta-conjunto . . . 36
4.2.1 Dimensionamento do Meta-conjunto . . . 37
4.2.2 Distribuição de Desempenho no Meta-conjunto . . . 37
4.2.3 Análise por Componente do Pipeline . . . 38
4.2.4 Embedding Distribucional . . . 40
4.2.5 Visualização UMAP dos Embeddings . . . 41
4.2.6 Matriz de Similaridade dos Embeddings . . . 42
4.3 Resultados da Fase 2 – Avaliação LOO do Preditor de Ranking . . . 43
4.3.1 Desempenho por Valor de k . . . 43
4.3.2 Distribuição das Métricas por Fold . . . 44
4.4 Análise do Kendall Tau Ponderado . . . 45
4.4.1 Comparação entre τ e τ_w . . . 45
4.5 Correlação Estatística: Embeddings e Qualidade de Ranqueamento . . 46
4.6 Demonstração Ilustrativa: Recomendação para Conjunto de Dados Retido 47
4.7 Discussão dos Resultados . . . 48
4.7.1 Comparação com Baselines Triviais . . . 48
4.7.2 Comparação com Baselines e Literatura . . . 49
4.7.3 Interpretação da Aparente Contradição entre NDCG e Spearman . . . 50
4.7.4 Embedding distribucional vs. Dataset2Vec . . . 50
4.7.5 Limitações . . . 51
5 Conclusão . . . 52
5.1 Síntese do Trabalho . . . 52
5.2 Atendimento dos Objetivos . . . 52
5.3 Limitações e Avaliação Crítica . . . 53
5.4 Trabalhos Futuros . . . 54

5.5 Considerações Finais 54

REFERÊNCIAS 56

1 INTRODUÇÃO

A seleção do algoritmo mais adequado para um determinado problema de aprendizado de máquina (ML, *Machine Learning*) é, reconhecidamente, um dos maiores desafios enfrentados por profissionais da área. Isso ocorre porque não existe um algoritmo universalmente superior para todos os tipos de dados, conforme reforçado pelo Teorema *No Free Lunch* Wolpert e Macready (1997), que demonstra formalmente que, em média sobre todos os problemas possíveis, qualquer dois algoritmos de otimização ou aprendizado têm desempenho equivalente — implicando que a superioridade de um algoritmo em um domínio necessariamente vem acompanhada de inferioridade em outro. As características intrínsecas dos dados — como dimensionalidade, balanceamento de classes, distribuição e presença de ruídos — impactam diretamente o desempenho dos modelos, tornando inviável a adoção de uma abordagem única e generalista.

Nesse cenário, o desenvolvimento de pipelines eficientes em ML exige uma combinação cuidadosa de três componentes fundamentais: representação, avaliação e otimização, conforme proposto por Domingos (2017). A representação define o espaço de hipóteses possível; a avaliação, os critérios que distinguem bons modelos de ruins; e a otimização, os métodos de busca que encontram o modelo mais adequado. Tradicionalmente, a construção desses pipelines depende fortemente da expertise humana, que realiza sucessivas tentativas e ajustes manuais.

Com o objetivo de reduzir esse esforço, surgem as abordagens de *AutoML* (*Automated Machine Learning*), que automatizam desde a seleção de algoritmos até o ajuste de hiperparâmetros. Deste modo, como destacam Schoenfeld (2020) e Vanschoren (2018), a eficácia dos sistemas de AutoML se amplia quando são utilizados modelos de meta-aprendizado, capazes de aprender a partir da experiência acumulada em múltiplos problemas anteriores.

O meta-aprendizado, ou aprendizado para aprender, estrutura-se na coleta de metadados e na extração de *meta-características* dos conjuntos de dados — atributos que capturam aspectos estatísticos, estruturais e informacionais dos dados Schoenfeld, Seppi e Giraud-Carrier (2022). Com isso, torna-se possível treinar metamodelos capazes de prever, de maneira informada, quais algoritmos ou pipelines possuem maior chance de sucesso em um novo problema.

Entretanto, apesar dos avanços, boa parte dos sistemas de AutoML e meta-aprendizado concentra-se na escolha do modelo e na configuração dos hiperparâmetros, desconsiderando um fator crítico: a qualidade do *pré-processamento dos dados*. Dados inconsistentes, com escalas divergentes, alta presença de *outliers* ou problemas de codificação categórica podem comprometer todo o pipeline, independentemente da qualidade do algoritmo selecionado.

Diante desse cenário, este trabalho propõe e avalia uma abordagem de meta-aprendizado para **ranqueamento e recomendação de pipelines** de classificação. A proposta consiste em: (i) construir um meta-conjunto a partir da execução sistemática de 60 pipelines — combinações de escaladores, seletores de atributos e classificadores — sobre 201 conjuntos de dados obtidos do OpenML (VANSCHOREN *et al.*, 2014), representando cada conjunto por uma

representação estatística distribucional de 64 dimensões, também referida neste trabalho como *embedding* estatístico por ser um vetor numérico de dimensão fixa que resume propriedades do conjunto de dados sem treinamento neural[1]; e (ii) treinar um preditor k-NN multi-saída, avaliado por validação LOO no nível de conjuntos de dados, capaz de estimar o ranking de desempenho esperado dos pipelines para um conjunto inédito.

1.1 Objetivos

Este trabalho tem como objetivo geral desenvolver e avaliar um sistema de meta-aprendizado baseado em embeddings estatísticos de conjuntos de dados para ranqueamento e recomendação de pipelines de classificação, capaz de identificar, a partir das características do conjunto de dados de entrada, quais configurações de pré-processamento e classificação apresentam maior probabilidade de sucesso preditivo. Para alcançar esse objetivo geral, foram definidos os seguintes objetivos específicos:

- Avaliar, por meio da construção de um meta-conjunto — repositório de conhecimento contendo descrições de pipelines, embeddings dos conjuntos de dados e rótulos de desempenho obtidos por validação cruzada estratificada —, se metadados de desempenho e meta-características de conjuntos de dados carregam sinal suficiente para diferenciar boas e más combinações de pré-processamento e classificação;
- Construir e avaliar um modelo preditivo capaz de estimar, a partir de metacaracterísticas dos dados, o ranking de adequação de diferentes configurações de pré-processamento e classificação.

1.2 Justificativa

A relevância deste trabalho assenta-se em duas lacunas complementares dos sistemas atuais de AutoML. A primeira é a tendência de focar exclusivamente na seleção do algoritmo ou na otimização de hiperparâmetros, desconsiderando a avaliação conjunta das etapas de pré-processamento — escalonamento, seleção de atributos e classificação — como um pipeline integrado Schoenfeld, Seppi e Giraud-Carrier (2022). Problemas como escalonamento inadequado ou seleção de atributos prejudicial podem comprometer completamente o desempenho do modelo, independentemente da qualidade do classificador escolhido Domingos (2017).

A segunda lacuna refere-se à forma de avaliação: a maioria dos sistemas de AutoML trata a recomendação como seleção do *melhor* pipeline (top-1), perdendo a informação ordinal completa. Schoenfeld, Seppi e Giraud-Carrier (2022) demonstraram que ranquear pipelines com

[1] O conceito de *embedding* de conjuntos de dados e a distinção entre representações neurais e estatísticas são detalhados na Seção 2.7.

base na similaridade de meta-características dos conjuntos de dados produz recomendações superiores às estimativas de desempenho absoluto. Contudo, as métricas de avaliação utilizadas na literatura raramente penalizam de forma diferenciada os erros nas primeiras posições do ranking — exatamente onde o impacto prático é maior para um usuário de AutoML que avalia os primeiros pipelines sugeridos.

Este trabalho preenche essas lacunas ao: (i) avaliar pipelines como combinações completas de pré-processamento e classificação; (ii) adotar ranqueamento como paradigma de recomendação; e (iii) utilizar Kendall Tau ponderado e NDCG como métricas complementares para avaliar a qualidade das primeiras posições do ranking, onde o impacto prático da recomendação é maior. O meta-conjunto construído, com 201 conjuntos de dados e 60 pipelines avaliados por validação cruzada estratificada, constitui ainda uma contribuição empírica de escala superior à maioria dos estudos da área Vanschoren (2018).

1.3 Estrutura do Trabalho

Este trabalho está organizado em cinco capítulos:

- **Capítulo 1 – Introdução:** apresenta o contexto, o problema de pesquisa, os objetivos, a justificativa e a estrutura geral do trabalho;
- **Capítulo 2 – Referencial Teórico:** discute os fundamentos de aprendizado de máquina, AutoML, meta-aprendizado, meta-características para caracterização de conjuntos de dados, aprendizado ativo, estrutura de pipelines, componentes de pré-processamento e classificação, embeddings de conjuntos de dados e métricas de ranqueamento, e revisa os principais trabalhos relacionados nas áreas de representação de conjuntos de dados por embeddings, sistemas de recomendação de algoritmos e pipelines via meta-aprendizado, e análise de complexidade de fronteiras de decisão;
- **Capítulo 3 – Materiais e Métodos:** descreve os materiais utilizados, a metodologia experimental em duas fases — construção do meta-conjunto e meta-aprendizado por ranqueamento —, o preditor k-NN multi-saída, o protocolo de validação Leave-One-Out e as métricas de avaliação de ranking;
- **Capítulo 4 – Experimentos e Resultados:** apresenta os resultados da Fase 1 (caracterização e análise do meta-conjunto construído) e da Fase 2 (avaliação LOO do preditor), incluindo análise do Kendall Tau ponderado, correlação estatística entre embedding e qualidade de ranqueamento, e uma demonstração ilustrativa em conjunto de dados retido;
- **Capítulo 5 – Conclusão:** sintetiza as contribuições do trabalho, verifica o atendimento dos objetivos, discute limitações identificadas e propõe direções para pesquisas futuras.

2 REFERENCIAL TEÓRICO

O aprendizado de máquina moderno envolve uma série de decisões interdependentes que impactam diretamente a eficácia dos modelos, desde o pré-processamento dos dados até a escolha e configuração dos algoritmos. A crescente complexidade desses processos impulsionou o surgimento do *AutoML*, cujo objetivo é automatizar etapas críticas de um pipeline preditivo. No entanto, para que essa automação seja eficiente, é necessário compreender os fundamentos que orientam tais decisões.

Este capítulo apresenta os principais conceitos que embasam a proposta deste trabalho, abordando inicialmente os fundamentos do Aprendizado de Máquina e a estrutura dos sistemas de *AutoML*. Em seguida, discute-se o meta-aprendizado como estratégia para reutilizar conhecimento adquirido em problemas anteriores, otimizando escolhas futuras. Também são exploradas as meta-características, essenciais para caracterizar conjuntos de dados e treinar metamodelos eficazes, bem como o aprendizado ativo, que visa otimizar o processo de rotulagem. Por fim, são apresentadas estratégias para seleção e estruturação de pipelines, os componentes de pré-processamento e classificação utilizados no experimento, as representações vetoriais de conjuntos de dados e as métricas de ranqueamento adotadas na avaliação.

2.1 Aprendizado de Máquina e AutoML

A aprendizagem de máquina envolve sistemas que aprendem automaticamente a partir de dados, sem necessidade de programação explícita do modelo. Domingos (2017) destaca que, para não se perder na variedade de algoritmos disponíveis, deve-se encarar o aprendizado como a combinação de três componentes principais: representação, avaliação e otimização. Nesse contexto:

- **Representação:** define a linguagem formal dos classificadores que o sistema pode aprender, isto é, o espaço de hipóteses permitido. Se um classificador não está nesse espaço, ele não pode ser obtido pelo aprendizado;
- **Avaliação:** função objetivo utilizada para distinguir classificadores bons de maus (por exemplo, acurácia, erro quadrático). Em geral, há uma função interna de otimização distinta da métrica externa desejada;
- **Otimização:** método de busca no espaço de hipóteses para encontrar o classificador de maior pontuação. Técnicas comuns incluem busca combinatória, algoritmos evolucionários, gradiente etc.

A automação desses componentes é o foco do AutoML. Sistemas de AutoML têm se mostrado eficazes ao empregar *metamodelos* — modelos treinados sobre o histórico de desempenho de algoritmos em execuções anteriores, e não diretamente sobre os dados do problema-

alvo — de forma *offline* (isto é, previamente, a partir de execuções já realizadas em outras bases) para guiar a seleção de algoritmos e parâmetros, abordagem detalhada na Seção 2.2. Em particular, Schoenfeld (2020) observa que sistemas AutoML obtêm melhor desempenho usando metamodelos offline para estimar o desempenho de modelos. Vanschoren (2018) aponta também que o meta-aprendizado pode acelerar e aprimorar o projeto de pipelines de ML, substituindo abordagens manuais por soluções aprendidas de forma orientada por dados.

2.2 Meta-aprendizado

O meta-aprendizado, ou "aprender a aprender"Vilalta e Drissi (2002), é a ciência de observar sistematicamente como diferentes abordagens de aprendizado de máquina performam em múltiplas tarefas, e então aprender com essa experiência (metadados) para resolver novas tarefas mais rapidamente, de modo a construir sistemas de aprendizado autoadaptativos que melhoram seu viés indutivo dinamicamente com a experiência acumulada Brazdil *et al.* (2008). Em outras palavras, utiliza-se o histórico de desempenho de algoritmos em uma coleção de bases de dados para treinar um *metamodelo* (modelo treinado sobre metadados de execuções anteriores, e não diretamente sobre os dados do problema-alvo) que pode prever o melhor algoritmo ou configuração para um novo problema. Adicionalmente, Schoenfeld, Seppi e Giraud-Carrier (2022) mostraram que, em vez de tratar modelos como caixas-pretas, pode-se explorar metadados granulares dos pipelines para melhorar a previsão de desempenho.

As abordagens de meta-aprendizado podem ser *offline* ou *online*. Em métodos *offline*, diversos pipelines são executados previamente em coleções de conjunto de dados e os resultados formam o meta conjunto usado para treinar o metamodelo. Sistemas AutoML como o Auto-sklearn Feurer *et al.* (2015) e o SmartML Maher e Sakr (2019) ilustram esse caso, utilizando os resultados de múltiplas execuções em bases conhecidas para guiar a busca em novas bases. Em contrapartida, métodos *online*, como o TPOT (OLSON *et al.*, 2016), atualizam o aprendizado à medida que geram pipelines no próprio problema de interesse.

Além disso, benchmarks específicos como o *meta conjunto*, proposto por Triantafillou *et al.* (2020), foram desenvolvidos para avaliar modelos de few-shot learning e meta-aprendizado em cenários mais realistas e desafiadores. O *meta conjunto* é composto por um conjunto diversificado de conjuntos de dados que permite avaliar a capacidade dos meta-modelos de generalizar para novos domínios e tarefas com poucos exemplos. A proposta introduz variações no número de classes, tamanho de conjuntos de suporte e testes com desbalanceamento, promovendo uma avaliação mais robusta dos métodos de meta-aprendizado.

Em todos os casos, o objetivo é transferir conhecimento entre tarefas: utilizar metadados e meta características coletados em problemas anteriores para reduzir o tempo e esforço de modelagem em novos problemas.

2.3 Meta-características e Caracterização de Conjuntos de Dados

A caracterização de dados por meio de *meta características* é essencial para o meta-aprendizado. Meta características são atributos que descrevem propriedades estatísticas e estruturais de um conjunto de dados, e servem de entrada para os metamodelos. Por exemplo, Schoenfeld, Seppi e Giraud-Carrier (2022) extraíram 72 meta características de cada base de dados, incluindo categorias simples, estatísticas, informações teóricas, *landmarking* e baseadas em modelo. Esses grupos podem ser definidos genericamente da seguinte forma:

- **Simples:** propriedades elementares do conjunto, como número de instâncias, número de atributos, proporção de classes;
- **Estatísticas:** medidas como médias, variâncias e correlações dos atributos;
- **Informacionais:** métricas de teoria da informação, como entropia das classes e entropia dos atributos;
- **Landmarking:** desempenho de algoritmos simples e rápidos (por exemplo, taxa de erro de 1-vizinho mais próximo, árvore de decisão rasa) no conjunto de dados;
- **Baseadas em modelo:** características derivadas de modelos treinados, como profundidade média de árvores de decisão, coeficientes de regressão, etc.

Essas meta características capturam informações relevantes que permitem medir similaridades entre conjuntos de dados e orientar o aprendizado Feurer, Springenberg e Hutter (2015). Muitas pesquisas utilizam repositórios de dados públicos; por exemplo, o conjunto de 194 bases usado por Schoenfeld, Seppi e Giraud-Carrier (2022) provém do programa D3M (*Data-Driven Discovery of Models*), iniciativa da DARPA voltada à automação da ciência de dados Lippmann, Campbell e Campbell (2016), e inclui conjuntos de dados originados de repositórios como o OpenML Vanschoren *et al.* (2014) e o UCI Machine Learning Repository (DUA; GRAFF, 2019). Cada instância do meta conjunto consiste no vetor de metacaracterísticas do conjunto de dados e na estrutura do pipeline avaliado, associado ao respectivo desempenho (por exemplo, medida F1) obtido naquela base. Esse meta conjunto (meta características + descrição de pipeline + desempenho) é então utilizado para treinar metamodelos capazes de prever ou ranquear pipelines em novos conjuntos de dados.

2.4 Aprendizado Ativo

O aprendizado ativo é uma abordagem em que o algoritmo seleciona interativamente quais instâncias devem ser rotuladas para melhorar seu desempenho com poucos dados rotulados Cohn, Atlas e Ladner (1994). Settles (2009) descreve que a ideia central do aprendizado

ativo é permitir que o sistema escolha os exemplos a serem rotulados, de forma que se atinja maior precisão com um número reduzido de etiquetas. Em outras palavras, um modelo com capacidade de consulta pode alcançar desempenho elevado utilizando apenas os exemplos mais informativos. Esse paradigma é motivado por cenários em que há abundância de dados não rotulados e o custo de rotular é alto.

Para implementar o aprendizado ativo, definem-se estratégias de consulta que estimam a utilidade de rotular cada instância disponível. Abordagens comuns incluem amostragem por incerteza (rotular as instâncias nas quais o modelo está menos confiante), seleção por comitê (vários modelos votam nas instâncias mais controversas) e redução de erro esperado.

No contexto de meta-aprendizado para AutoML, o aprendizado ativo é relevante como extensão futura: ao invés de avaliar todos os 60 pipelines em cada novo conjunto de dados, um sistema poderia selecionar ativamente quais pipelines avaliar primeiro com base na incerteza do metamodelo, reduzindo o custo computacional da construção do meta-conjunto. Doucet *et al.* (2025) e Sener e Savarese (2018) exemplificam essa abordagem ao utilizar *embeddings* para guiar a seleção de exemplos informativos no espaço de características — princípio análogo ao uso de *embeddings* de conjuntos de dados para identificar quais avaliações de pipeline são mais informativas para o sistema de recomendação.

2.5 Seleção e estrutura de *pipelines*

Em sistemas AutoML, um *pipeline* de aprendizado de máquina consiste na composição de múltiplos passos de pré-processamento e modelagem que transformam os dados até a predição final. Em geral, pipelines são representados como grafos acíclicos direcionados (DAGs), podendo incluir estruturas paralelas como comitês. Essa flexibilidade permite representar desde um pipeline simples (uma sequência linear de transformação e classificador) até configurações complexas (ex.: três sub pipelines com combinação por votação majoritária).

Entretanto, o número de pipelines possíveis cresce rapidamente com o número de algoritmos e passos disponíveis. Como observado por Schoenfeld, Seppi e Giraud-Carrier (2022), as diversas maneiras de combinar algoritmos em pipelines criam um espaço de busca *não escalonável* de possíveis fluxos a serem considerados. Diante disso, é útil empregar estratégias de pré-seleção ou ranqueamento de pipelines candidatos para priorizar a execução daqueles com maior potencial (especialmente quando recursos computacionais são limitados). Por exemplo, abordagens tradicionais de AutoML, como Auto-sklearn (FEURER *et al.*, 2015) e SmartML (MAHER; SAKR, 2019), tratam o pipeline como uma caixa-preta e utilizam apenas o desempenho observado e a similaridade de meta características do conjunto de dados para ranquear

modelos. Outras soluções simplificam a estrutura do pipeline, por exemplo considerando apenas sequências lineares e usando uma rede LSTM para estimar valor[1].

Em contraste, trabalhos mais recentes exploram explicitamente a estrutura do pipeline. Schoenfeld (2020) propõe um metamodelo baseado em rede neural dinamicamente montada, capaz de modelar DAGs arbitrários do pipeline. Schoenfeld, Seppi e Giraud-Carrier (2022) validam empiricamente essa abordagem criando um grande meta conjunto (cerca de 400.000 instâncias de aproximadamente 4.000 pipelines avaliados em 194 bases) e mostram que metamodelos que consideram metadados granulares de cada pipeline superam aqueles que usam representações simplificadas. Concretamente, eles demonstraram que metamodelos que utilizam toda a estrutura do pipeline fornecem estimativas de desempenho mais precisas, e que ranquear pipelines baseado na similaridade das meta características do conjunto de dados resulta em recomendações superiores às baseadas em estimativas de desempenho absolutas.

Em resumo, a seleção de pipelines em AutoML combina conhecimento do domínio (via meta-características de dados) e exploração da estrutura dos fluxos de processamento. Esse campo de pesquisa continua em desenvolvimento, mas já existe consenso de que considerar explicitamente a granularidade e a estrutura dos pipelines é crucial para sistemas AutoML eficientes e escaláveis.

2.6 Componentes do Arsenal Experimental

Como a metodologia deste trabalho avalia combinações de escalonamento, seleção de atributos e classificação, é necessário explicitar o papel dos operadores que compõem o arsenal experimental. Todos os componentes foram implementados por meio da biblioteca *scikit-learn*, referência consolidada para experimentação reprodutível em aprendizado de máquina com Python (PEDREGOSA *et al.*, 2011).

No estágio de escalonamento, foram considerados três transformadores e uma alternativa sem transformação. O `StandardScaler` centraliza cada atributo pela média e o reescala pelo desvio padrão, tornando os atributos comparáveis em termos de variância. O `MinMaxScaler` reescala os valores para um intervalo fixo, usualmente $[0,1]$. O `RobustScaler` utiliza mediana e intervalo interquartil, sendo menos sensível a valores extremos. Esses operadores são relevantes porque algoritmos baseados em distância, margem ou otimização numérica, como k-NN, SVM e MLP, podem ser fortemente afetados por diferenças de escala entre atributos.

No estágio de seleção ou redução de atributos, foram utilizados PCA, `SelectKBest` e ausência de seleção. A PCA projeta os dados em componentes ortogonais que concentram a maior variância possível, sendo uma técnica clássica de redução de dimensionalidade (JOL-

[1] LSTM: rede neural recorrente que aprende dependências de longo prazo em sequências por meio de um mecanismo de portas que regula o fluxo de informação em uma célula de memória interna Hochreiter e Schmidhuber (1997).

LIFFE, 2002). O `SelectKBest`, neste trabalho, utiliza o critério `f_classif`, baseado no teste F da ANOVA, para manter os atributos mais associados ao alvo; trata-se de uma estratégia de filtragem supervisionada de atributos (GUYON; ELISSEEFF, 2003). A alternativa `None` preserva todos os atributos originais, permitindo avaliar quando a redução dimensional prejudica ou favorece o desempenho.

No estágio de classificação, foram avaliados cinco algoritmos. O RF combina múltiplas árvores de decisão treinadas sobre amostras e subconjuntos de atributos, reduzindo variância e aumentando robustez em dados tabulares (BREIMAN, 2001). A SVM busca hiperplanos de máxima margem e, com kernel RBF, modela fronteiras não lineares (CORTES; VAPNIK, 1995). O k-NN é um método baseado em instâncias que classifica uma nova amostra pela vizinhança mais próxima no espaço de atributos (COVER; HART, 1967). A MLP é uma rede neural feedforward capaz de modelar relações não lineares por camadas ocultas (BISHOP, 2006). Por fim, a DT particiona recursivamente o espaço de atributos por regras de decisão, oferecendo interpretabilidade e baixo custo computacional (QUINLAN, 1986).

2.7 Embeddings de Conjuntos de Dados e Métricas de Ranking

Esta seção apresenta os dois fundamentos diretamente empregados na proposta deste trabalho: as representações vetoriais de conjuntos de dados conhecidas como *embeddings*, utilizadas para caracterizar os conjuntos de dados do meta-conjunto, e as métricas empregadas para avaliar a qualidade de um ranqueamento de pipelines.

2.7.1 Embeddings de Conjuntos de Dados

Uma limitação das meta-características tradicionais é que cada conjunto de dados é representado por um único vetor de estatísticas globais — médias, variâncias, entropia das classes —, o que descarta a variabilidade interna da distribuição dos atributos. Para superar essa limitação, abordagens mais recentes propõem *embeddings* de conjuntos de dados: vetores densos de dimensão fixa que capturam propriedades representativas do conjunto de dados de forma invariante ao número de atributos e instâncias.

Jomaa, Schmidt-Thieme e Grabocka (2021) propõem o Dataset2Vec, um *embedding* neural baseado em arquitetura DeepSet que representa o conjunto de dados como um conjunto hierárquico de pares preditores-alvo. O modelo é treinado com uma tarefa auxiliar de similaridade entre conjuntos de dados e produz vetores de 64 dimensões cujas proximidades no espaço euclidiano correlacionam-se com a similaridade de desempenho de algoritmos. Os autores identificam quatro critérios desejáveis para embeddings de conjuntos de dados, a saber: (D1) independência de esquema — aplicável a qualquer conjunto sem restrição de dimensionalidade; (D2) expressividade — capacidade de distinguir conjuntos com características de

aprendizado distintas; (D3) escalabilidade — calculável sem treinamento para novos conjuntos; (D4) correlação com meta-alvos — os vetores devem predizer desempenho de algoritmos.

Uma alternativa sem necessidade de treinamento consiste nos *embeddings* estatísticos distribucionais: ao invés de uma única estatística global por atributo, calcula-se a distribuição desta estatística sobre todos os atributos do conjunto de dados via percentis, produzindo um vetor de dimensão fixa que satisfaz os critérios D1 e D3 e se aproxima de D2 em tarefas tabulares. Esta abordagem é análoga ao MF2[2] de Feurer, Springenberg e Hutter (2015), porém com maior riqueza representacional por capturar distribuições e não apenas médias globais.

2.7.2 Métricas de Avaliação de Ranking

Na tarefa de ranqueamento de pipelines, a qualidade das predições é medida por métricas que comparam a ordenação predita com a ordenação real. As principais métricas utilizadas na literatura de meta-aprendizado são:

Coeficiente de Correlação de Spearman (ρ): mede a correlação monotônica entre os postos atribuídos aos scores preditos e reais. Valores próximos de 1 indicam que o ranking predito preserva a ordem correta dos pipelines; valores próximos de zero indicam ausência de correlação Spearman (1904).

Kendall Tau (τ): mede a fração líquida de pares de pipelines corretamente ordenados. Dado um par (i, j), o par é concordante se o ranking predito e o real concordam sobre qual pipeline é superior; é discordante caso contrário. O valor de τ é a diferença entre pares concordantes e discordantes, normalizada pelo total de pares Kendall (1938).

Kendall Tau Ponderado (τ_w): extensão do Kendall Tau que atribui pesos maiores às inversões de ordem que ocorrem no topo do ranking. Implementado em `scipy.stats.weightedtau`, o peso de cada par de posições (i,j) é inversamente proporcional ao produto de seus postos no ranking real, de modo que erros envolvendo os melhores pipelines são penalizados mais severamente do que erros nas posições inferiores.

NDCG (*Normalized Discounted Cumulative Gain*): métrica proveniente da área de recuperação de informação, que pondera a relevância dos itens recuperados pelo logaritmo da posição na lista. Em meta-aprendizado, o NDCG penaliza explicitamente pipelines de alto desempenho recomendados em posições distantes do topo.

A adoção conjunta dessas quatro métricas permite avaliar o sistema tanto em termos de correlação global (Spearman e Kendall Tau) quanto de precisão nas posições de maior impacto prático. Nesse segundo grupo, Kendall Tau ponderado e NDCG são métricas complementa-

[2] O MF2 é um conjunto de 46 meta-características estáticas — incluindo medidas de *landmarking*, componentes principais (PCA) e proporção de valores ausentes — originalmente proposto por Reif, Shafait e Dengel (2012) e empregado por Feurer, Springenberg e Hutter (2015) para estimar a similaridade entre conjuntos de dados na inicialização de otimização bayesiana de hiperparâmetros.

res: a primeira penaliza inversões pareadas no topo do ranking, enquanto a segunda mede a relevância acumulada das primeiras posições.

2.8 Trabalhos Relacionados

Esta seção apresenta os principais trabalhos relacionados à proposta desta pesquisa, organizados em três eixos temáticos: (i) abordagens para extração de meta-características e representação de conjuntos de dados; (ii) sistemas de recomendação de algoritmos e pipelines de aprendizado de máquina via meta-aprendizado; e (iii) estratégias de avaliação de complexidade de fronteiras de decisão e aprendizado ativo. Ao final, é apresentada uma síntese comparativa que posiciona a presente proposta em relação ao estado da arte.

2.8.1 Representação de Conjuntos de Dados e Meta-características

A qualidade dos metamodelos em sistemas de meta-aprendizado depende diretamente da capacidade de representar conjuntos de dados de forma compacta e informativa. Historicamente, duas correntes principais se desenvolveram: a engenharia de meta-características estáticas e a aprendizagem de meta-características por redes neurais.

Vanschoren *et al.* (2014) introduzem o OpenML, uma plataforma colaborativa que centraliza conjuntos de dados, experimentos e resultados de algoritmos em um repositório público e estruturado. Além de prover acesso unificado a centenas de conjuntos de dados anotados, o OpenML define uma infraestrutura para coleta sistemática de meta-dados — acurácias, tempos de execução, hiperparâmetros utilizados — que viabiliza a construção de grandes meta-conjuntos. Esta plataforma constitui a principal fonte de dados do presente trabalho, tendo fornecido os 201 conjuntos utilizados nos experimentos.

Schoenfeld, Seppi e Giraud-Carrier (2022) investigam a construção de meta-conjuntos de grande escala para otimização de pipelines, criando uma base com aproximadamente 400 mil instâncias derivadas de 4 mil pipelines avaliados em 194 conjuntos do programa D3M. O trabalho demonstra empiricamente que metamodelos que consideram metadados granulares de toda a estrutura do pipeline superam abordagens que utilizam apenas representações simplificadas. Os autores mostram, ainda, que o ranqueamento baseado em similaridade de meta-características fornece recomendações superiores às estimativas de desempenho absoluto, convergindo diretamente com a abordagem adotada neste TCC.

Jomaa, Schmidt-Thieme e Grabocka (2021) propõem o *Dataset2Vec*, um extrator de meta-features baseado em uma arquitetura DeepSet que representa conjuntos de dados como conjuntos hierárquicos de pares preditores-alvo. O modelo é treinado com uma tarefa auxiliar de aprendizagem de similaridade entre conjuntos de dados (*dataset similarity learning*), que determina se dois subconjuntos amostrados provêm do mesmo conjunto. Os resultados expe-

rimentais com 120 conjuntos UCI para a tarefa de otimização de hiperparâmetros demonstram que as meta-features aprendidas pelo Dataset2Vec superam meta-features estáticas tradicionais (MF1 de Wistuba, Schilling e Schmidt-Thieme (2016) com 22 dimensões; MF2 de Feurer, Springenberg e Hutter (2015) com 46 dimensões) e o *Neural Statistician* (Edwards e Storkey (2017)) em termos de correlação de Spearman com os alvos de desempenho.

O presente trabalho diferencia-se do Dataset2Vec em dois aspectos fundamentais: (1) ao invés de embeddings neurais, utiliza-se um embedding distribucional de 64 dimensões, produzido sem necessidade de treinamento prévio, com propriedades análogas ao MF2 mas com maior riqueza por capturar a *distribuição* das meta-features por atributo (não apenas médias globais); (2) a tarefa meta-nível é o *ranqueamento de pipelines* (não apenas otimização de hiperparâmetros), sobre um meta-conjunto de 201 conjuntos de dados. Esta escolha de embedding estático garante a propriedade de independência de esquema (**D1** na taxonomia do Dataset2Vec), escalabilidade sem treinamento (**D3**) e reprodutibilidade, ao custo de expressividade potencialmente menor (**D2**) em relação a abordagens neurais.

2.8.2 Sistemas de Recomendação de Algoritmos e Pipelines

Schoenfeld (2020) propõem um metamodelo baseado em rede neural dinamicamente configurável, capaz de modelar DAGs arbitrários de pipelines. O sistema representa cada operador do pipeline com uma subrrede específica e combina as representações por uma função de pooling global. Esta abordagem permite estimar o desempenho de configurações de pipeline inéditas sem avaliação direta, tornando a exploração do espaço de busca mais eficiente. A arquitetura supera metamodelos com representações simplificadas de pipeline, reforçando a importância de considerar a estrutura interna dos fluxos de processamento.

Golshanrad *et al.* (2021) apresentam o sistema MEGA (*Meta-learning Ensemble for Genetic Algorithm*), um método de combinação de classificadores que emprega meta-aprendizado e algoritmo genético para selecionar a melhor combinação de classificadores para um conjunto de dados desconhecido. O sistema extrai meta-características dos conjuntos, usa um algoritmo genético para encontrar combinações eficazes e, na fase de inferência, aplica um k-NN ponderado para recomendar combinações ideais. O uso do k-NN como metamodelo de inferência é compatível com a abordagem adotada neste trabalho, onde também se emprega k-NN para predição de rankings de pipelines.

Pereira, Santos e Carvalho (2021) investigam a recomendação de algoritmos de seleção de atributos por meta-aprendizado, modelando a tarefa como um problema supervisionado. O trabalho emprega k-NN, SVM, árvores de decisão e florestas aleatórias como meta-aprendizes, com meta-características extraídas de 28 meta-conjuntos do repositório ASlib. Os autores demonstram que técnicas de redução de dimensionalidade (PCA, seleção por variância, Chi-quadrado, ANOVA-F) permitem reduzir em mais de 80% o número de meta-características

sem degradar o desempenho preditivo, evidenciando a robustez de abordagens baseadas em similaridade para recomendação.

Wang, Song e Zhu (2021) propõem um método multi-rótulo de recomendação de algoritmos de classificação baseado em aprendizado por comitê. O modelo mapeia meta-características de 1090 problemas de classificação para o conjunto de algoritmos adequados, combinando múltiplos tipos de meta-características em modelos-base com integração ponderada. Os experimentos demonstram superioridade em métricas de ranking (perda de ordenação e *Precision@m*) sobre modelos individuais, reforçando o valor de abordagens de comitê para recomendação.

Khan *et al.* (2023) descrevem o AutoFE-Sel, sistema de meta-aprendizado para recomendação de algoritmos de seleção de subconjunto de atributos. A abordagem utiliza a métrica EARR (*Adjusted Ratio of Ratios*), que combina acurácia, tempo de execução e número de atributos selecionados para comparar algoritmos. Experimentos com 125 conjuntos de dados e 12 algoritmos candidatos demonstram que o AutoFE-Sel supera abordagens tradicionais (MCFA, IBL-Ranking, Fuzzy-Sim) em acurácia, *hit-ratio* e *F-measure*, atingindo acerto superior a 92%.

2.8.3 Avaliação de Fronteiras de Decisão e Aprendizado Ativo

Varshney e Ramamurthy (2015) utilizam topologia persistente para caracterizar a complexidade das fronteiras de decisão em classificadores supervisionados, construindo complexos simpliciais a partir de amostras com rótulos distintos. Os números de Betti obtidos pela homologia persistente correlacionam-se com o desempenho preditivo dos modelos de SVM, auxiliando na seleção de hiperparâmetros. Embora a abordagem topológica seja distinta da proposta neste TCC, o princípio de usar características intrínsecas dos dados para orientar a seleção de modelos é compartilhado.

Ramamurthy, Varshney e Mody (2018) expandem a análise topológica propondo os complexos *labeled Čech* e *labeled Vietoris-Rips* para estimar a complexidade topológica da fronteira de decisão (DBTC). Experimentos em MNIST, FashionMNIST e CIFAR-10 mostram que a correspondência entre a DBTC do modelo e a do conjunto de dados favorece a seleção de modelos pré-treinados, evidenciando que propriedades estruturais dos dados podem guiar a escolha de algoritmos.

Peng (2020) apresentam uma revisão abrangente sobre meta-aprendizado, sistematizando métodos baseados em otimização, modelo, métrica e inferência Bayesiana. O trabalho discute como o aprendizado meta-episódico permite generalização eficiente para novas tarefas com dados rotulados limitados, especialmente em cenários de *few-shot learning* (aprendizado a partir de pouquíssimos exemplos rotulados por classe), formalizando a notação de meta-tarefas $\tau_i \sim \mathcal{P}(\mathcal{T})$ com conjuntos de suporte $\mathcal{D}_i^s$ e consulta $\mathcal{D}_i^q$.

Doucet *et al.* (2025) propõem o método TCM, que combina amostragem baseada em diversidade e incerteza para aprendizado ativo. O TCM realiza uma transição gradual de *TypiClust*

— que seleciona exemplos representativos por clustering em embeddings auto-supervisionados — para *Margin Sampling*, focado na incerteza do modelo. Os autores demonstram superioridade sobre BADGE, Coreset e ProbCover em CIFAR-10, CIFAR-100 e ISIC2019, evidenciando o valor de combinar diversidade e incerteza na seleção de instâncias informativas. A utilização de embeddings para identificar exemplos representativos é análoga ao uso de embeddings de conjuntos de dados neste trabalho para identificar conjuntos similares no espaço de meta-aprendizado.

2.8.4 Síntese Comparativa

A Tabela 2 sintetiza os principais trabalhos relacionados em relação a quatro dimensões: tipo de representação dos dados, tarefa de meta-nível, tamanho do meta-conjunto e método de avaliação utilizado.

Tabela 2 – Comparativo entre trabalhos relacionados e a proposta deste TCC

Trabalho	Representação	Tarefa Meta-nível	Meta-conjunto	Avaliação
Schoenfeld et al. (2022)	Estrutura do pipeline (DAG)	Estimativa de desempenho	$\sim$400 k instâncias	Correlação de Spearman
Dataset2Vec (2021)	Embedding neural (DeepSet)	Otimização de hiperparâmetros	120 conjuntos UCI	Spearman (ranqueamento)
MEGA (2020)	Meta-características estáticas	Recomendação de comitês	Múltiplos conjuntos	Acurácia
AutoFE-Sel (2021)	Meta-características estáticas	Seleção de atributos	125 conjuntos	EARR, hit-ratio
Wang et al. (2021)	Meta-características multi-tipo	Recomendação multi-rótulo	1090 conjuntos	Ranking loss, Prec@m
Este TCC	**embedding distribucional 64D**	**Ranqueamento de pipelines**	**201 conjuntos / 60 pipelines**	**Spearman, Kendall (τ, τ_w), NDCG**

Fonte: Autoria própria (2026).

A análise comparativa revela que a presente proposta distingue-se dos trabalhos anteriores principalmente por três aspectos: (1) foco explícito no *ranqueamento completo de pipelines* (não apenas seleção do melhor ou estimativa pontual); (2) adoção conjunta de **Kendall Tau ponderado** (*weightedtau*) e NDCG como métricas complementares de topo, alinhando a avaliação às necessidades reais de AutoML, onde os primeiros pipelines recomendados têm maior impacto prático; e (3) análise estatística da correlação entre dimensões do embedding e qualidade de ranqueamento por conjunto de dados, investigando *quando* e *por que* o meta-aprendizado funciona melhor.

A escolha de uma representação estatística distribucional, referida ao longo do trabalho como embedding distribucional por produzir um vetor denso de dimensão fixa, foi motivada pela compatibilidade com Python 3.13+, onde TensorFlow não possui distribuição, e pelo objetivo de estabelecer um baseline reprodutível e publicável. Diferentemente de embeddings neurais como o Dataset2Vec, essa representação não é aprendida por otimização de uma função objetivo; ela se aproxima do MF2 de Feurer et al. (2015) em termos de engenharia de meta-características, mas captura distribuições de meta-features por atributo em vez de apenas estatísticas globais do conjunto de dados.

3 MATERIAIS E MÉTODOS

Este capítulo descreve os materiais, ferramentas computacionais, conjuntos de dados e métodos empregados no desenvolvimento desta pesquisa. Inicialmente são apresentados os recursos utilizados, seguidos de um panorama geral do processo experimental baseado em dois fluxogramas. Por fim, detalha-se cada etapa metodológica da Fase 1 e Fase 2 do experimento de meta-aprendizado para recomendação de pipelines de aprendizado de máquina.

3.1 Materiais Utilizados

Nesta seção, são apresentados os recursos computacionais, linguagens de programação, bibliotecas, ambientes de desenvolvimento e conjuntos de dados empregados ao longo da pesquisa. A seleção desses elementos foi guiada por critérios como aderência às práticas consolidadas na área de Aprendizado de Máquina (ML), disponibilidade em código aberto, escalabilidade e suporte à experimentação científica.

Antes de detalhar cada grupo de materiais individualmente, é oportuno oferecer uma visão geral do ecossistema tecnológico que fundamenta esta investigação. O Quadro 1 a seguir consolida os principais componentes utilizados, organizando-os por categoria, versão (quando aplicável) e justificativa de sua escolha. Esta apresentação permite ao leitor compreender, de forma estruturada, a base técnica sobre a qual os experimentos foram conduzidos.

Quadro 1 – Materiais, Ferramentas e Conjuntos de Dados Utilizados

Categoria	Nome Específico	Versão	Justificativa/Relevância
Linguagem	Python	3.13	Ecossistema robusto para AM, vastas bibliotecas
Ambiente de Desenvolvimento	Jupyter / VS Code	N/A	Experimentação interativa, gerenciamento de código
Biblioteca de AM	Scikit-learn	$\geq$ 1.4	Pipelines, pré-processamento, classificadores e métricas
Estatística Computacional	SciPy	$\geq$ 1.12	KDTree, correlações e métricas de ranqueamento
Meta-características	Implementação própria	N/A	Extração de embeddings estatísticos distribucionais
Manipulação de Dados	Pandas	$\geq$ 1.3	Estrutura *DataFrame* para dados tabulares
Computação Numérica	NumPy	$\geq$ 1.20	Arrays multidimensionais e operações matemáticas
Visualização	Matplotlib, Seaborn e umap-learn	N/A	Gráficos, mapas de calor e projeção UMAP
Fonte de Dados	OpenML	N/A	Repositório com conjuntos anotados e metadados
Recurso Computacional	PC local	Próprio	Execução dos experimentos das Fases 1 e 2

Fonte: Autoria própria (2026).

O quadro destaca a integração entre ferramentas consolidadas do ecossistema científico de Python e infraestrutura computacional local, evidenciando o cuidado na seleção de tecnologias que garantam reprodutibilidade, desempenho e flexibilidade. A seguir, cada categoria é explorada individualmente, de forma a detalhar seu papel específico no desenvolvimento dos experimentos desta pesquisa.

Os sites oficiais das principais ferramentas utilizadas são: Python[1], Jupyter[2], Visual Studio Code[3], scikit-learn[4], SciPy[5], NumPy[6], pandas[7], umap-learn[8] e OpenML[9].

3.1.1 Linguagens e Ambientes de Desenvolvimento

O projeto foi desenvolvido em Python (versão 3.13), utilizando como ambiente principal o Jupyter Notebook e, em etapas mais estruturadas, o Visual Studio Code. Essas ferramentas foram escolhidas devido à sua ampla adoção na comunidade científica e à compatibilidade com bibliotecas de aprendizado de máquina. O uso de controle de versão via `Git` e `GitHub` também foi empregado para rastreamento do código.

3.1.2 Bibliotecas e Frameworks

As principais bibliotecas utilizadas foram:

- `scikit-learn`: Para construção de pipelines, pré-processamento, validação cruzada e algoritmos base (RF, SVM, k-NN, MLP e DT) (PEDREGOSA *et al.*, 2011);
- `scipy`: Para a estrutura `KDTree`, correlações de Spearman e Kendall, e cálculo do Kendall Tau ponderado;
- `pandas` e `numpy`: Para manipulação e estruturação de dados tabulares e vetoriais;
- `matplotlib`, `seaborn` e `umap-learn`: Para geração dos gráficos, mapas de calor e projeção bidimensional dos embeddings.

3.1.3 Conjuntos de Dados

Os conjuntos de dados foram obtidos do repositório público OpenML (VANSCHOREN *et al.*, 2014), selecionando-se tarefas de classificação supervisionada que atendessem aos seguintes critérios: (i) entre 500 e 10.000 instâncias, de modo a equilibrar representatividade estatística e custo computacional; (ii) ao menos duas classes; e (iii) atributos predominantemente

[1] https://www.python.org/
[2] https://jupyter.org/
[3] https://code.visualstudio.com/
[4] https://scikit-learn.org/
[5] https://scipy.org/
[6] https://numpy.org/
[7] https://pandas.pydata.org/
[8] https://umap-learn.readthedocs.io/
[9] https://www.openml.org/

numéricos. Dos 243 *task IDs* configurados inicialmente, **201 conjuntos de dados** foram carregados com sucesso e tiveram ao menos um pipeline avaliado sem erros — número que supera os 120 conjuntos do UCI Repository utilizados por (JOMAA; SCHMIDT-THIEME; GRABOCKA, 2021) como referência na literatura para tarefas de meta-aprendizado. Os 42 conjuntos restantes foram descartados por falha no download ou por não atenderem aos critérios de qualidade estabelecidos.

A caracterização local dos 201 conjuntos carregados indica heterogeneidade suficiente para avaliar o ranqueamento de pipelines: o número de instâncias varia de 500 a 10.000, com mediana de 1.941; o número de atributos varia de 1 a 1.776, com mediana de 20; e a mediana da proporção da classe majoritária é 0,50. Apenas um conjunto apresentou valores ausentes após o carregamento, caso tratado pela rotina de pré-processamento. A Tabela 3 apresenta exemplos representativos do meta-conjunto.

Tabela 3 – Exemplos de conjuntos de dados utilizados no meta-conjunto

Task ID	Nome	Caracterização	Instâncias	Atributos	Classes	Classe majoritária
3	`kr-vs-kp`	Jogo de xadrez	3.196	36	2	0,522
11	`balance-scale`	Balanceamento físico	625	4	3	0,461
9910	`Bioresponse`	Classificação bioquímica	3.751	1.776	2	0,542
363713	`regensburg_pediatric_appendicitis`	Diagnóstico médico tabular	780	55	3	0,619

Fonte: Autoria própria (2026), com base nos dados carregados do OpenML..

3.1.4 Recursos Computacionais

Os experimentos foram executados integralmente em hardware local, em uma estação de trabalho pessoal com as seguintes especificações:

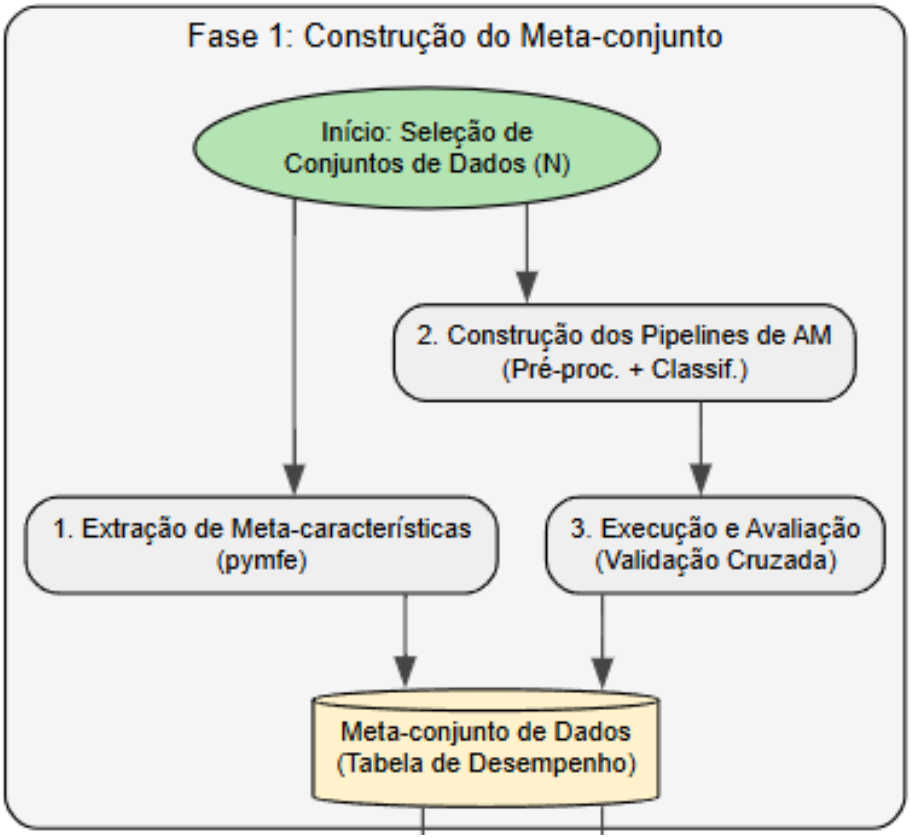
- Processador: AMD Ryzen 7 5700U com Gráficos Radeon (8 núcleos, 16 threads, 1,80 GHz base);
- Memória RAM: 12 GB DDR4;
- Armazenamento: SSD (Solid-State Drive), utilizado como cache dos conjuntos de dados e dos embeddings, garantindo operações de leitura e escrita de alta velocidade durante a manipulação dos dados;
- Sistema Operacional: Windows 10 Pro (*build* 19045);
- Ambiente: Python 3.13, com as bibliotecas `scikit-learn` ($\geq$ 1.4), `scipy` ($\geq$ 1.12), `numpy`, `pandas` e `umap-learn`.

Esta configuração mostrou-se adequada para a execução dos experimentos: a Fase 1 (execução de 60 pipelines $\times$ 201 conjuntos de dados), intensiva em CPU e I/O, foi paralelizada sobre os núcleos do processador; e a Fase 2 (validação LOO do preditor k-NN), de custo computacional linear no número de conjuntos de dados, foi concluída em poucos minutos.

3.2 Métodos

O processo experimental foi dividido em duas fases principais. A Fase 1 consiste na construção de um meta-conjunto de dados, onde pipelines são aplicados a conjuntos de dados previamente caracterizados. A Fase 2 realiza o meta-aprendizado e fornece a recomendação de pipelines adequados.

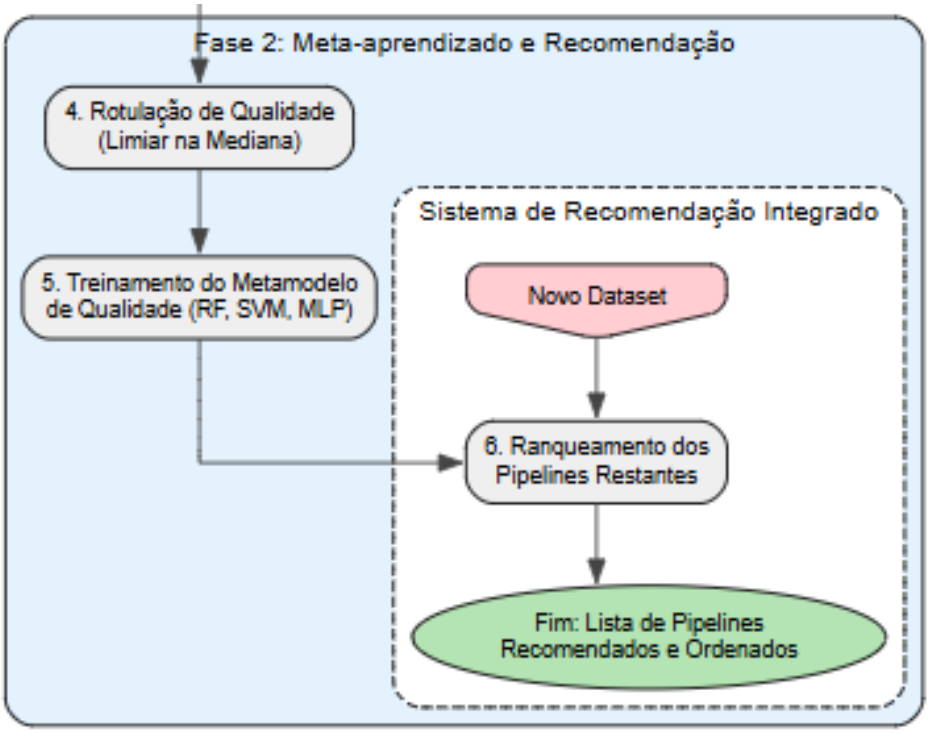
Figura 1 – Fluxograma do processo experimental: Fase 1 - Construção do meta-conjunto



Fonte: Autoria própria (2026).

O processo inicia com a seleção de conjuntos de dados, seguida da extração do embedding estatístico distribucional, da execução de pipelines (pré-processamento + classificador) e da avaliação via validação cruzada. Os dados resultantes são organizados no meta-conjunto com informações como: embedding do conjunto de dados (X_{emb}), configuração do pipeline e desempenho obtido (Y_{perf}).

Figura 2 – Fluxograma do processo experimental: Fase 2 - Meta-aprendizado e recomendação no processo experimental



Fonte: Autoria própria (2026).

Na Fase 2, o meta-conjunto é utilizado para treinar e avaliar um preditor de ranqueamento multi-saída. Para um novo conjunto de dados, o sistema localiza os conjuntos mais similares no espaço de embeddings e estima a acurácia esperada de cada pipeline, gerando uma lista ordenada de recomendações.

A seguir, cada fase do processo experimental é pormenorizada. A Fase 1 foca na criação da base de conhecimento – o meta-conjunto de dados –, que é a fundação para a abordagem de meta-aprendizado. A Fase 2 utiliza essa base para treinar os metamodelos e, finalmente, gerar as recomendações de pipelines, que constitui o objetivo central desta pesquisa.

3.2.1 Fase 1 – Construção do Meta-conjunto

A caracterização dos conjuntos de dados é realizada por um extrator de embeddings estatísticos distribucionais implementado neste trabalho. O método calcula, para cada atributo numérico do conjunto de dados, um conjunto de estatísticas descritivas (média, desvio padrão, mínimo, máximo, mediana, percentis P_{25} e P_{75}, assimetria e curtose) e, em seguida, agrega a distribuição dessas estatísticas sobre todos os atributos via percentis. O vetor é composto por 45 meta-features distribucionais (9 estatísticas $\times$ 5 agregadores), 10 propriedades globais do conjunto e 9 posições preenchidas com zero, totalizando 64 dimensões antes da normalização para norma unitária. Essa etapa gera o vetor $\mathbf{x} \in \mathbb{R}^{64}$, que é utilizado como entrada para o

preditor k-NN[10]. A escolha de um embedding estatístico sem treinamento neural garante aplicabilidade imediata a novos conjuntos de dados sem necessidade de retreinamento, atendendo ao critério D3 do Dataset2Vec (JOMAA; SCHMIDT-THIEME; GRABOCKA, 2021).

Os pipelines são compostos por sequências de operadores de pré-processamento seguidos por um classificador. A modularidade dos componentes permite gerar múltiplas combinações, cujas estruturas são registradas de forma detalhada para compor o histórico do experimento.

Cada pipeline é avaliado por meio de validação cruzada estratificada de 5 folds sobre os conjuntos de dados. A métrica de desempenho adotada é a **acurácia balanceada** (*balanced accuracy*), definida como a média aritmética da sensibilidade (recall) de cada classe, conforme a Equação (1) Brodersen *et al.* (2010):

$$\text{BA} = \frac{1}{C} \sum_{c=1}^{C} \frac{TP_c}{TP_c + FN_c} \tag{1}$$

onde C é o número de classes, TP_c é o número de verdadeiros positivos e FN_c o de falsos negativos para a classe c. Para $C = 2$, a acurácia balanceada equivale à média aritmética de sensibilidade e especificidade. A escolha desta métrica em detrimento da acurácia simples é motivada pela presença de conjuntos de dados desbalanceados no meta-conjunto: a acurácia simples favorece artificialmente classificadores que preveem apenas a classe majoritária, enquanto a acurácia balanceada garante que o desempenho em todas as classes seja igualmente considerado. O desempenho médio sobre os 5 folds compõe o valor Y_{perf} armazenado na matriz de acurácias.

Ao final da Fase 1, o principal artefato gerado é o meta-conjunto de dados, que consolida o conhecimento empírico obtido a partir da execução sistemática de diversos pipelines sobre uma ampla variedade de conjuntos de dados. Este meta-conjunto é estruturado de forma que cada instância representa um par formado por um conjunto de dados e uma configuração de pipeline, contendo as seguintes informações:

- **Embedding estatístico do conjunto de dados (X_{emb}):** representa as características intrínsecas do conjunto de dados por meio de estatísticas distribucionais, servindo como entrada para o preditor de ranqueamento;
- **Configuração do pipeline:** descreve a sequência de operadores de pré-processamento e o classificador utilizados, bem como seus respectivos hiperparâmetros;
- **Desempenho do pipeline (Y_{perf}):** armazena os resultados das métricas obtidas durante a validação cruzada, como acurácia balanceada e F1-score.

[10] O k-vizinhos mais próximos (k-NN) é um algoritmo de aprendizado baseado em instâncias que estima a saída para uma nova entrada a partir dos k exemplos de treinamento mais próximos no espaço de características, segundo uma métrica de distância Cover e Hart (1967).

3.2.2 Fase 2 – Meta-aprendizado por Ranqueamento

Com o meta-conjunto construído, a Fase 2 realiza a **predição de ranking** de pipelines para novos conjuntos de dados. Esta abordagem trata a recomendação como um problema de ranqueamento (em vez de classificação binária), o que preserva toda a informação ordinal de desempenho e permite ao usuário selecionar quantos pipelines desejar da lista recomendada.

3.2.2.1 Preditor k-NN Multi-saída

O metamodelo adotado é um **preditor k-vizinhos mais próximos** (k-NN) multi-saída, que utiliza uma estrutura `KDTree`[11] para busca eficiente de vizinhos no espaço de embeddings. A KDTree é uma árvore binária de particionamento espacial que organiza os pontos por planos de corte em cada dimensão, reduzindo a complexidade de busca de $O(n \cdot d)$ (força bruta) para $O(d \cdot \log n)$ em média, onde n é o número de conjuntos de dados e $d = 64$ é a dimensão do embedding. Para um novo conjunto de dados D_{new} com embedding $\mathbf{x}_{new} \in \mathbb{R}^{64}$, o preditor identifica os k conjuntos de dados mais similares no espaço de embeddings e estima a acurácia esperada de cada pipeline P_j como a média das acurácias observadas nos vizinhos, conforme a Equação (2) — generalização multi-saída da regressão k-NN clássica, em que a predição é dada pela média das observações dos k vizinhos mais próximos Altman (1992):

$$\hat{A}(P_j) = \frac{1}{k} \sum_{D_i \in \mathcal{N}_k(D_{new})} A_{i,j} \tag{2}$$

onde $\mathcal{N}_k(D_{new})$ denota o conjunto dos k conjuntos de dados mais próximos de D_{new} no espaço de embeddings e $A_{i,j}$ é a acurácia balanceada do pipeline P_j no conjunto de dados D_i. A similaridade entre conjuntos de dados é medida pela distância euclidiana no espaço de embeddings. A recomendação final consiste no ranking dos 60 pipelines ordenados por $\hat{A}(P_j)$ de forma decrescente.

3.2.2.2 Validação por Leave-One-Out

A avaliação do preditor é realizada por **Leave-One-Out (LOO) no nível de conjuntos de dados**: para cada conjunto D_i no meta-conjunto, o preditor é treinado nos $n - 1$ conjuntos restantes e avalia suas predições sobre o conjunto excluído. Este protocolo garante que cada avaliação simula o cenário real de inferência — um conjunto de dados genuinamente inédito —, fornecendo estimativas não enviesadas da capacidade de generalização.

[11] A KDTree (*k-dimensional tree*) é uma estrutura de árvore binária que particiona recursivamente o espaço de busca por planos de corte alinhados aos eixos, permitindo consultas de vizinhança eficientes em espaços multidimensionais Bentley (1975).

3.2.2.3 Métricas de Avaliação de Ranking

A qualidade do ranqueamento predito em relação ao ranqueamento real foi avaliada por quatro métricas complementares:

- **Coeficiente de Correlação de Spearman (ρ)** (SPEARMAN, 1904): mede a correlação monotônica entre os postos dos scores preditos e reais. $\rho = 1$ indica ordenação perfeita; $\rho = 0$, ausência de correlação linear de postos;
- **Kendall Tau (τ)** (KENDALL, 1938): mede a fração líquida de pares de pipelines corretamente ordenados, conforme a Equação (3):

$$\tau = \frac{P - Q}{\binom{n}{2}} \tag{3}$$

  onde P é o número de pares concordantes e Q o de pares discordantes. $\tau = 1$ indica todos os pares na ordem correta; $\tau = -1$, ordem invertida;
- **Kendall Tau Ponderado (τ_w)**: implementado via `scipy.stats.weightedtau`, atribui pesos maiores às inversões de ordem que ocorrem no topo do ranking (melhores pipelines), penalizando erros de recomendação que têm maior impacto prático. Formalmente, o peso de cada par de posições (i, j) é inversamente proporcional ao produto de seus postos no ranking real;
- **NDCG** (*Normalized Discounted Cumulative Gain*): implementado via `sklearn.metrics.ndcg_score`, penaliza explicitamente pipelines relevantes classificados em posições inferiores, definido pela Equação (4):

$$\text{NDCG} = \frac{\text{DCG}}{\text{IDCG}}, \quad \text{DCG} = \sum_{i=1}^{n} \frac{2^{rel_i} - 1}{\log_2(i+1)} \tag{4}$$

  onde rel_i é a relevância do pipeline na posição i (acurácia balanceada real) e IDCG é o DCG do ranking ideal (ordenação perfeita). NDCG $= 1$ indica ranking perfeito; valores próximos de 1 indicam que os pipelines de maior relevância estão próximos do topo. Assim, τ_w e NDCG são tratados como métricas complementares de topo: o primeiro avalia inversões pareadas ponderadas, enquanto o segundo avalia a relevância acumulada das posições iniciais.

As métricas são reportadas como média e desvio padrão sobre todos os folds LOO válidos (folds com variância não nula no vetor de acurácias reais). Para a seleção do hiperparâmetro k, avalia-se $k \in \{1, 3, 5, 7, 10\}$ e o valor com maior Kendall médio é selecionado.

3.2.2.4 Análise de Correlação Estatística

Como análise complementar, investiga-se a **correlação estatística entre o Kendall Tau ponderado por conjunto de dados e as dimensões do embedding**. Para cada dimensão $d \in \{1, \ldots, 64\}$ do espaço de embeddings, calcula-se o coeficiente de Spearman entre $\{\tau_w^{(i)}\}_{i=1}^n$ (scores de qualidade de ranqueamento por conjunto de dados) e $\{x_d^{(i)}\}_{i=1}^n$ (valor da dimensão d no embedding de cada conjunto). Esta análise visa identificar quais aspectos da representação do conjunto de dados predizem a qualidade do meta-aprendizado, fornecendo interpretabilidade ao sistema.

4 EXPERIMENTOS E RESULTADOS

Este capítulo descreve os experimentos realizados para validar a abordagem proposta de meta-aprendizado para ranqueamento de pipelines. Inicialmente são apresentados a configuração experimental e o protocolo de avaliação. Em seguida, são reportados os resultados da construção do meta-conjunto (Fase 1) e da avaliação do preditor de ranking (Fase 2), incluindo a análise do Kendall Tau ponderado, a correlação estatística entre embeddings e qualidade de ranqueamento, e uma demonstração ilustrativa do sistema em operação. O capítulo encerra com uma discussão dos resultados em perspectiva com a literatura.

4.1 Configuração Experimental

Esta seção consolida as decisões experimentais que operacionalizam a metodologia descrita no capítulo anterior: o conjunto de pipelines avaliados, a fonte dos dados e o protocolo de validação utilizado para medir a capacidade de generalização do preditor de ranqueamento.

4.1.1 Arsenal de Pipelines

O arsenal de pipelines foi definido como o produto cartesiano de quatro grupos de operadores modulares:

- **Escalonamento (4):** `StandardScaler`, `MinMaxScaler`, `RobustScaler` e ausência de escalonamento (`None`);
- **Seleção de atributos (3):** `PCA` (componentes principais), `SelectKBest` com critério `f_classif` (ANOVA F, selecionando os $k = \min(10, p)$ melhores atributos) e ausência de seleção (`None`);
- **Classificadores (5):** `RandomForestClassifier` (RF), `SVC` (SVM), k-vizinhos mais próximos (k-NN), Rede Neural Multicamadas (MLP) e Árvore de Decisão (DT).

O total de combinações é $4 \times 3 \times 5 = 60$ pipelines distintos. Cada pipeline é avaliado com validação cruzada estratificada de 5 folds, usando acurácia balanceada como métrica de desempenho. A fundamentação dos componentes utilizados é apresentada na seção 2.6. A Tabela 4 apresenta as combinações em forma de resumo.

Tabela 4 – Arsenal de 60 pipelines: componentes disponíveis por estágio

Estágio	Opções	Quantidade
Escalonamento	StandardScaler, MinMaxScaler, RobustScaler, None	4
Seleção de atributos	PCA, SelectKBest, None	3
Classificador	RandomForest, SVM, kNN, MLP, DecisionTree	5
Total (produto)		**60**

Fonte: Autoria própria (2026).

4.1.2 Fonte de Dados

Os conjuntos de dados foram obtidos do repositório OpenML (VANSCHOREN *et al.*, 2014), selecionando tarefas de classificação supervisionada com as seguintes restrições:

- Entre 500 e 10.000 instâncias (equilíbrio entre representatividade e custo computacional);
- Ao menos duas classes;
- Atributos predominantemente numéricos.

De 243 *task IDs* configurados inicialmente, **201 conjuntos de dados** foram carregados com sucesso e tiveram ao menos um pipeline avaliado sem erros. Os 42 conjuntos restantes foram descartados por falha no download ou por não atenderem às restrições estabelecidas. A caracterização estatística e exemplos representativos desses conjuntos são apresentados na Tabela 3. Ressalta-se que este número supera os 120 conjuntos UCI utilizados por (JOMAA; SCHMIDT-THIEME; GRABOCKA, 2021) como referência na literatura para tarefas de meta-aprendizado.

4.1.3 Protocolo de Validação

A validação é realizada por **Leave-One-Out (LOO) no nível de conjuntos de dados**: em cada fold, um conjunto de dados é excluído do meta-conjunto, o preditor k-NN é treinado nos 200 conjuntos restantes e avaliado no conjunto excluído. Este procedimento gera 201 folds de validação independentes. Folds em que todos os pipelines obtiveram acurácia idêntica (variância nula) são descartados automaticamente, pois não fornecem informação ordinal.

4.2 Resultados da Fase 1 – Construção do Meta-conjunto

Esta seção apresenta os resultados obtidos na Fase 1 do experimento, abrangendo o dimensionamento do meta-conjunto construído, a distribuição de desempenho dos pipelines

avaliados, a análise por componente (escalonadores, seletores de atributos e classificadores) e a caracterização dos embeddings distribucionais gerados para os 201 conjuntos de dados, incluindo sua visualização via UMAP e a matriz de similaridade resultante.

4.2.1 Dimensionamento do Meta-conjunto

O meta-conjunto resultante possui as seguintes dimensões:

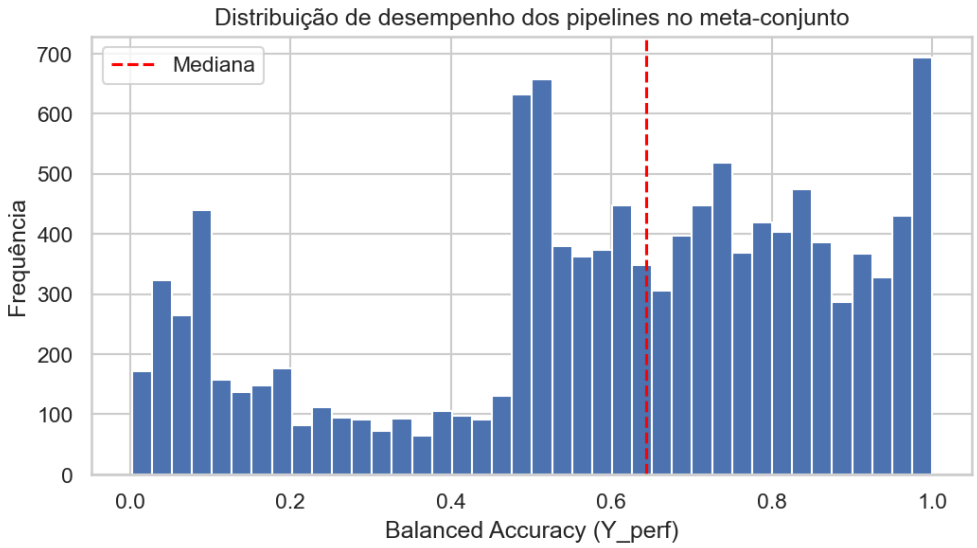
- **Matriz de embeddings** $\mathbf{X}_{emb} \in \mathbb{R}^{201\times64}$: cada linha é o vetor de embedding de 64 dimensões de um conjunto de dados;
- **Matriz de acurácias** $\mathbf{Y}_{acc} \in \mathbb{R}^{201\times60}$: entrada (i,j) é a acurácia balanceada média (5-fold) do pipeline j no conjunto i;
- **Valores ausentes:** 172 células ($< 1{,}5\%$ do total) correspondentes a pipelines que falharam em determinados conjuntos por questões de compatibilidade (ex.: SVM com dados de alta dimensionalidade sem pré-seleção).

O meta-conjunto, com $201 \times 60 = 12.060$ avaliações de pipeline, constitui a base de conhecimento sobre a qual o meta-aprendizado é realizado.

4.2.2 Distribuição de Desempenho no Meta-conjunto

A Figura 3 apresenta o histograma das 12.060 acurácias balanceadas registradas na matriz $\mathbf{Y}_{acc}$. A distribuição é multimodal: observa-se um pico próximo a zero, associado a pipelines inadequados para determinados conjuntos de dados (especialmente conjuntos multiclasse altamente desbalanceados), e concentrações adicionais entre 0,5 e 1,0, com mediana em torno de $0{,}63$. Esta assimetria positiva é esperada: a maioria dos pipelines avaliados com hiperparâmetros padrão produz acurácias moderadas a boas, enquanto os melhores pipelines atingem acurácias próximas de 1,0 em conjuntos com fronteiras de decisão simples.

Figura 3 – Distribuição de acurácias balanceadas na matriz $\mathbf{Y}_{acc}$ do meta-conjunto ($n = 12.060$)



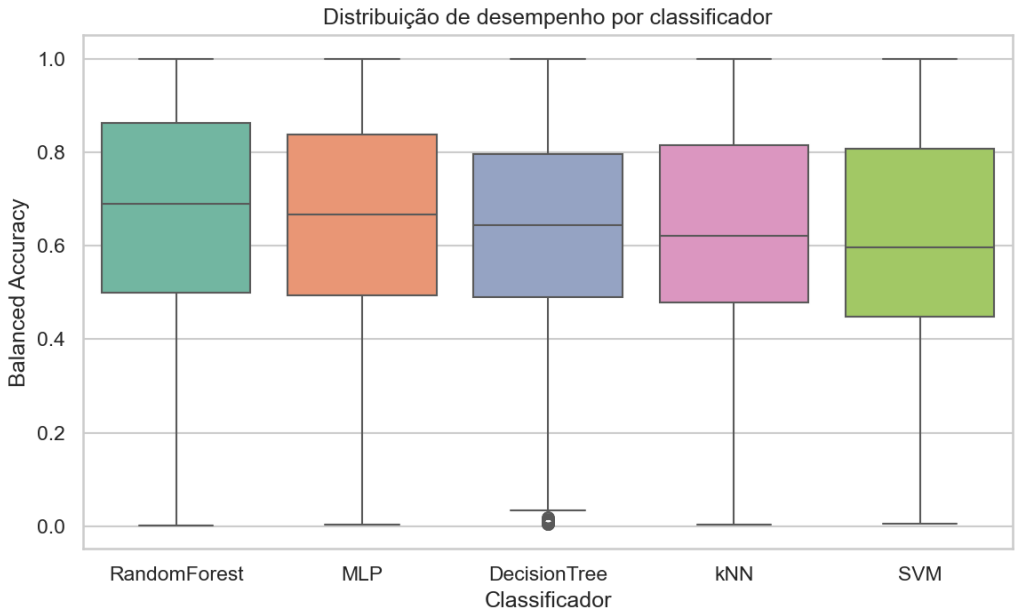
Fonte: Autoria própria (2026).

4.2.3 Análise por Componente do Pipeline

Para compreender quais componentes do arsenal contribuem mais para o desempenho, analisa-se a acurácia balanceada média desagregada por classificador, por combinação escalonador–classificador e por combinação seletor–classificador.

A Figura 4 apresenta a distribuição de acurácias por tipo de classificador, agregando todos os escaladores e seletores. O *Random Forest* obtém a maior mediana ($\approx 0{,}70$), seguido pelo MLP ($\approx 0{,}67$). O *Decision Tree*, apesar de acurácias máximas comparáveis, apresenta maior dispersão e um *outlier* próximo a zero, indicando maior sensibilidade ao conteúdo do conjunto de dados.

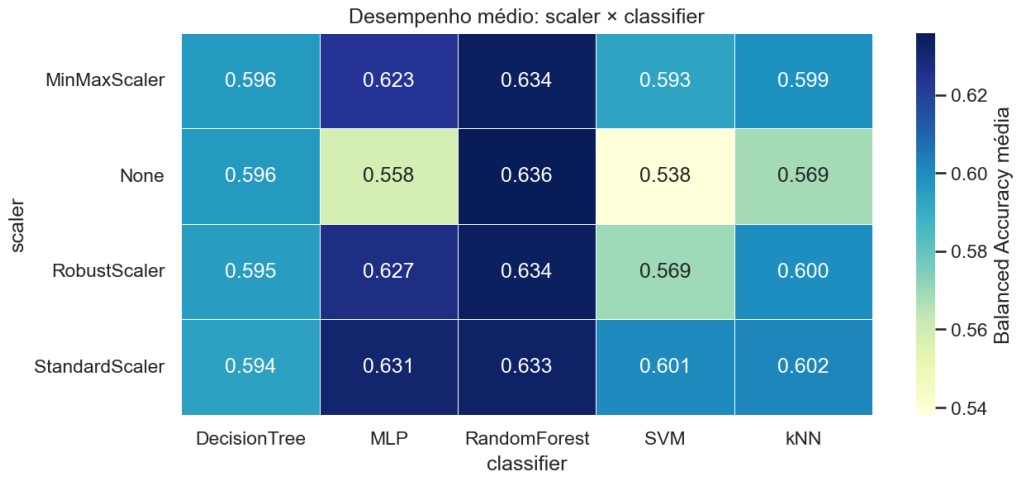
Figura 4 – Distribuição de acurácias balanceadas por classificador (todos os 201 conjuntos de dados)



Fonte: Autoria própria (2026).

As Figura 5 e Figura 6 apresentam mapas de calor da acurácia balanceada média por combinação de componentes. Na Figura 5, observa-se que a escolha do escalador tem impacto reduzido: todos os quatro escaladores (`MinMaxScaler`, `None`, `RobustScaler`, `StandardScaler`) produzem acurácias médias muito próximas para um mesmo classificador, com variação inferior a 0,01. O *Random Forest* destaca-se como o classificador mais robusto à escolha do escalador ($0{,}633$–$0{,}636$), enquanto o SVM apresenta a maior sensibilidade ($0{,}538$ sem escalador *vs.* $0{,}601$ com `StandardScaler`).

Figura 5 – Mapa de calor da acurácia balanceada média por escalador e classificador

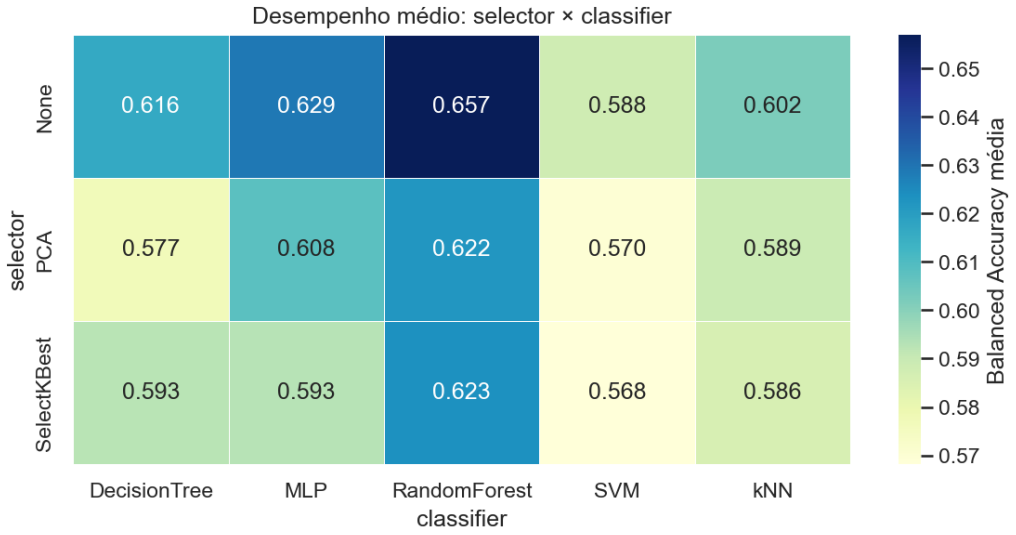


Fonte: Autoria própria (2026).

Na Figura 6, o impacto do seletor de atributos é mais pronunciado. Ausência de seleção (`None`) produz consistentemente os melhores resultados para todos os classificadores,

com *Random Forest* atingindo $0{,}657$ — o valor médio mais alto de toda a análise combinatorial. A aplicação de PCA degrada sistematicamente o desempenho em comparação com a ausência de seleção, com queda média de $3{,}5$ pontos percentuais. O SelectKBest apresenta desempenho intermediário, sugerindo que a compressão dimensional via componentes principais prejudica mais a representação dos dados para a maioria dos classificadores do que a seleção supervisionada de atributos.

Figura 6 – Mapa de calor da acurácia balanceada média por seletor de atributos e classificador



Fonte: Autoria própria (2026).

Esses resultados revelam que o *Random Forest* sem seleção de atributos é a configuração de classificador mais robusta na média, o que é consistente com a literatura (FERNÁNDEZ-DELGADO *et al.*, 2014), que demonstra que florestas aleatórias superam a maioria dos classificadores tabulares em benchmarks amplos. No entanto, a meta-aprendizagem opera justamente sobre as *exceções* a esta regra geral: o valor do sistema de recomendação está em identificar os conjuntos de dados para os quais configurações alternativas superam o *Random Forest* padrão.

4.2.4 Embedding Distribucional

Na ausência de uma distribuição compatível do TensorFlow para o ambiente Python 3.13+, o embedding foi gerado por um método estatístico distribucional desenvolvido neste trabalho como alternativa reprodutível ao Dataset2Vec. O método captura a *distribuição* de meta-features por atributo do conjunto de dados em três blocos:

1. **Bloco distribucional (45 dimensões):** para cada uma das 9 estatísticas por atributo (média, desvio padrão, mínimo, máximo, mediana, P_{25}, P_{75}, skewness e kurtosis), calcula-se a distribuição dessa estatística sobre todos os atributos do conjunto via [mín, P_{25}, média, P_{75}, máx], produzindo $9 \times 5 = 45$ meta-features;

2. **Bloco global (10 dimensões):** $\log(n)$, $\log(p)$, número de classes, proporção da classe majoritária, proporção da classe minoritária, entropia do alvo, proporção de atributos constantes, correlação média entre atributos, proporção média de valores únicos, razão p/n;

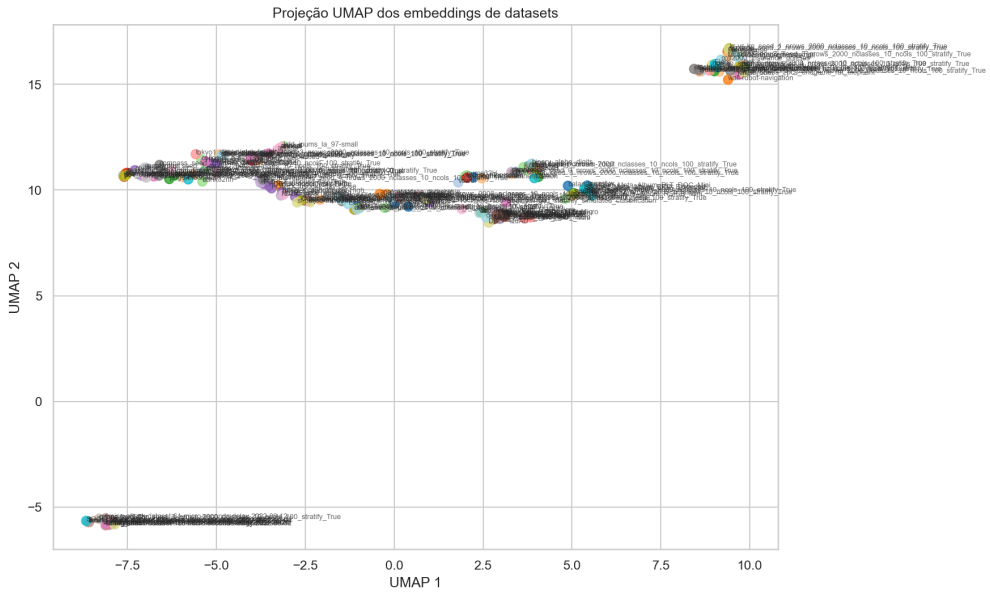
3. **Vetor final (64 dimensões):** os 55 valores anteriores são complementados com 9 posições preenchidas com zero, formando o vetor de 64 dimensões posteriormente normalizado para norma unitária.

Comparativamente, o Dataset2Vec de Jomaa, Schmidt-Thieme e Grabocka (2021) também produz vetores de 64 dimensões (configuração *units_h* = 32, expandido para 64 neste trabalho), aprendidos por uma rede DeepSet treinada com tarefa de similaridade entre conjuntos de dados. A principal vantagem do embedding neural é maior expressividade (critério **D2** da taxonomia D2V); a principal vantagem do embedding distribucional é que não requer treinamento (**D3**) e é aplicável sem restrição de versão do framework.

4.2.5 Visualização UMAP dos Embeddings

A Figura 7 exibe a projeção dos 201 embeddings de conjuntos de dados em 2D por meio do algoritmo UMAP (*Uniform Manifold Approximation and Projection*), configurado com `n_components=2` e `random_state=42`.

Figura 7 – Projeção UMAP dos embeddings de 201 conjuntos de dados no espaço bidimensional



Fonte: Autoria própria (2026).

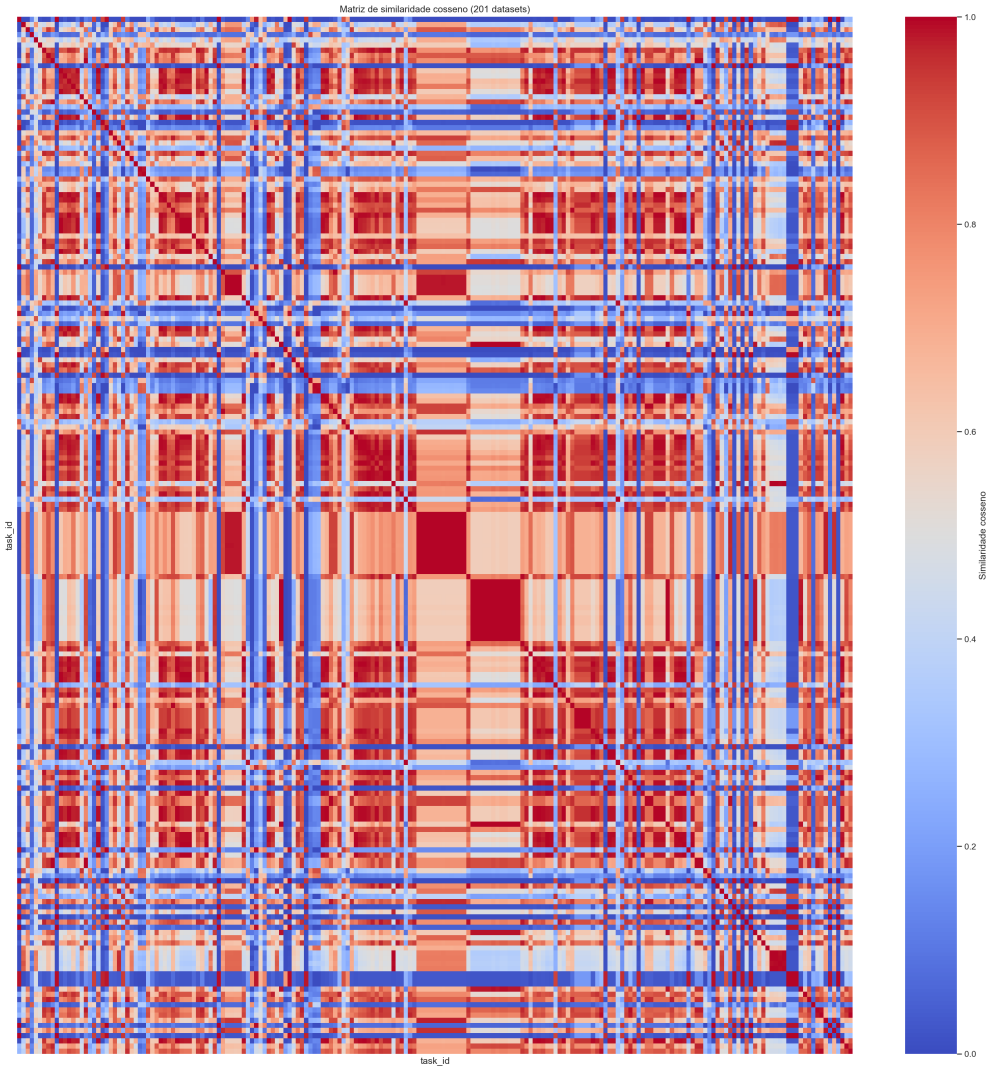
A projeção revela que os embeddings não são distribuídos uniformemente: observam-se agrupamentos locais que sugerem grupos de conjuntos de dados com características distribucionais similares, separados por regiões de menor densidade. Esta estrutura é compatível com

a hipótese central do meta-aprendizado por similaridade — conjuntos próximos no espaço de embeddings tendem a exibir padrões de desempenho similares entre os pipelines.

4.2.6 Matriz de Similaridade dos Embeddings

A Figura 8 apresenta a matriz de similaridade cosseno entre os 201 embeddings. Valores próximos de 1 (vermelho) indicam conjuntos altamente similares no espaço de representação; valores próximos de 0 (azul) indicam conjuntos com representações ortogonais.

Figura 8 – Matriz de similaridade cosseno entre os embeddings dos 201 conjuntos de dados



Fonte: Autoria própria (2026).

A presença de blocos diagonais de alta similaridade indica a existência de grupos de conjuntos de dados com propriedades distribucionais semelhantes, sugerindo que o embedding distribucional captura estrutura útil para meta-aprendizado baseado em similaridade.

4.3 Resultados da Fase 2 – Avaliação LOO do Preditor de Ranking

Esta seção apresenta os resultados da avaliação do preditor k-NN multi-saída por validação Leave-One-Out, incluindo o desempenho obtido para diferentes valores de k e a distribuição das métricas de ranqueamento (Spearman, Kendall Tau e NDCG) entre os folds de validação.

4.3.1 Desempenho por Valor de k

A Tabela 5 apresenta os resultados da avaliação LOO do preditor k-NN para $k \in \{1, 3, 5, 7, 10\}$. As métricas são reportadas como média $\pm$ desvio padrão sobre os folds válidos.

Tabela 5 – Resultados LOO do preditor k-NN para ranqueamento de pipelines (201 conjuntos de dados)

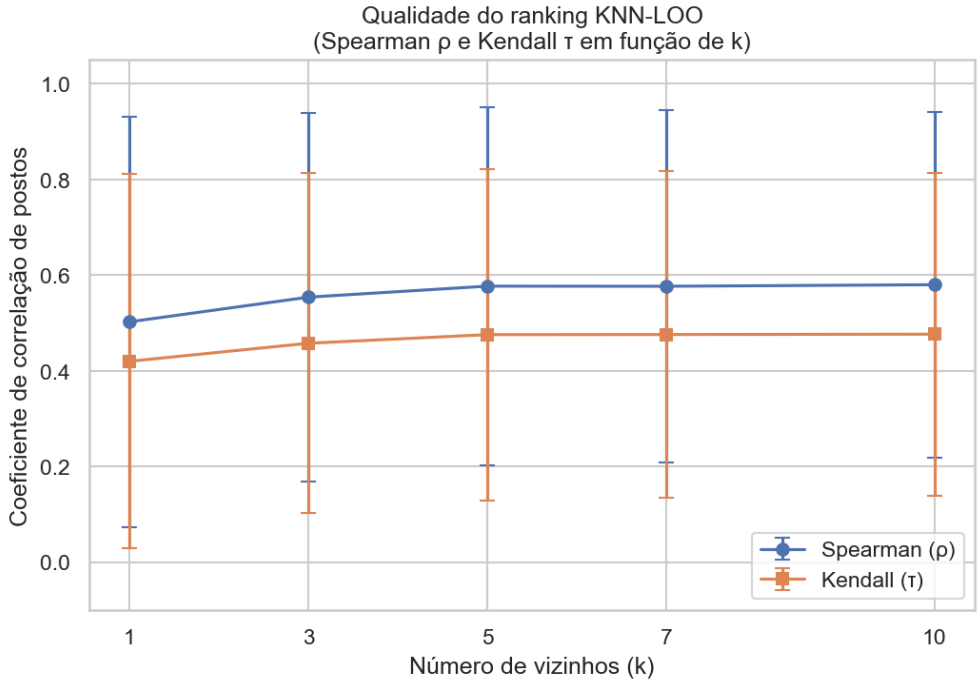
k	Spearman (ρ)	Kendall (τ)	Kendall pond. (τ_w)	NDCG	Folds válidos
1	$0{,}5025 \pm 0{,}4286$	$0{,}4200 \pm 0{,}3912$	$0{,}4630 \pm 0{,}4038$	$0{,}9833 \pm 0{,}0267$	197
3	$0{,}5541 \pm 0{,}3846$	$0{,}4577 \pm 0{,}3553$	$0{,}5034 \pm 0{,}3719$	$0{,}9830 \pm 0{,}0418$	198
5	$0{,}5771 \pm 0{,}3739$	$0{,}4757 \pm 0{,}3463$	$0{,}5231 \pm 0{,}3613$	$0{,}9841 \pm 0{,}0408$	198
7	$0{,}5768 \pm 0{,}3678$	$0{,}4759 \pm 0{,}3418$	$0{,}5233 \pm 0{,}3538$	$0{,}9844 \pm 0{,}0387$	198
10	$\mathbf{0{,}5801 \pm 0{,}3615}$	$\mathbf{0{,}4767 \pm 0{,}3379}$	$\mathbf{0{,}5222 \pm 0{,}3471}$	$\mathbf{0{,}9844 \pm 0{,}0384}$	**198**

Fonte: Autoria própria (2026).

O valor $k = 10$ obteve o maior Kendall médio ($\tau = 0{,}4767$) e foi selecionado como configuração ótima. A tendência geral é de melhora monotônica das métricas com o aumento de k, indicando que agregar informação de mais vizinhos reduz o ruído nas estimativas de desempenho, ao custo de uma eventual suavização da predição para conjuntos com características muito específicas.

A Figura 9 exibe graficamente o comportamento de Spearman e Kendall em função de k, com barras de erro correspondendo ao desvio padrão entre folds.

Figura 9 – Spearman e Kendall Tau em função de k (LOO sobre 201 conjuntos de dados)

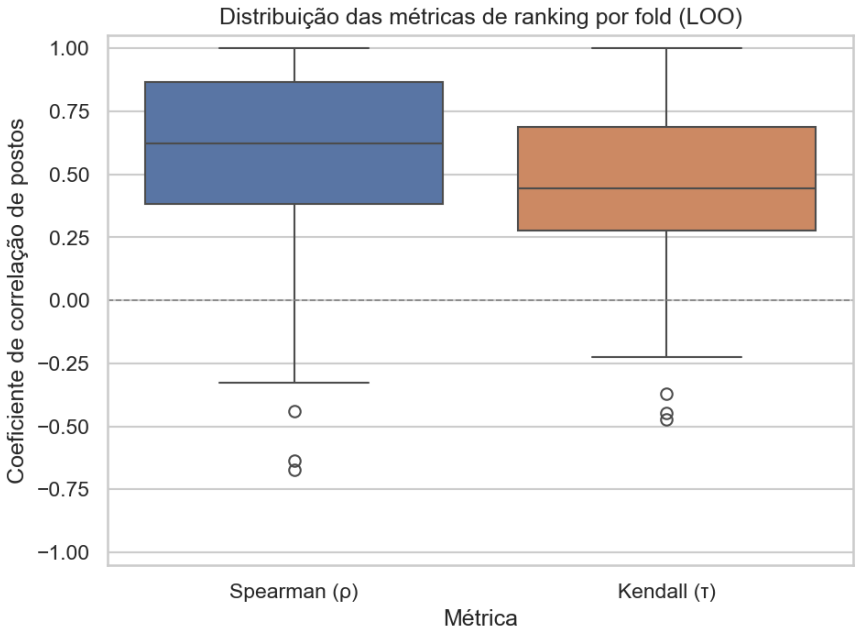


Fonte: Autoria própria (2026).

4.3.2 Distribuição das Métricas por Fold

A Figura 10 apresenta a distribuição de Spearman e Kendall sobre os 198 folds LOO válidos para $k = 10$. A distribuição multimodal revela a heterogeneidade dos conjuntos de dados: para alguns, o meta-aprendizado gera rankings quase perfeitos ($\rho > 0{,}9$); para outros, o preditor tem desempenho próximo ao aleatório.

Figura 10 – Distribuição de Spearman e Kendall Tau por fold LOO ($k = 10$, 198 folds)



Fonte: Autoria própria (2026).

Esta variabilidade elevada (desvio padrão $\approx 0{,}34$–$0{,}36$) é característica dos sistemas de meta-aprendizado baseados em similaridade: quando o conjunto de teste tem vizinhos similares no meta-conjunto, a recomendação é precisa; quando é atípico (sem vizinhos próximos), o preditor recorre a generalizações menos confiáveis.

4.4 Análise do Kendall Tau Ponderado

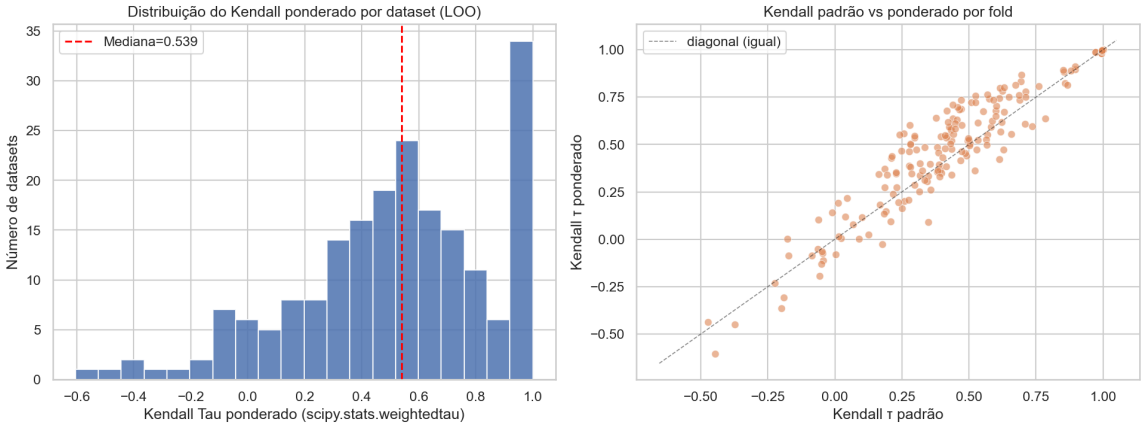
Esta seção aprofunda a análise dos resultados da Fase 2 sob a ótica do Kendall Tau ponderado — métrica que reflete a utilidade prática do ranqueamento ao penalizar com maior intensidade os erros cometidos no topo da lista de recomendações —, em articulação com o NDCG, que também privilegia as primeiras posições por desconto logarítmico. Em seguida, compara-se o desempenho do preditor com baselines triviais e com resultados reportados na literatura, discutem-se aparentes contradições entre métricas e apontam-se as principais limitações identificadas.

4.4.1 Comparação entre τ e τ_w

O Kendall Tau ponderado (τ_w) — implementado via `scipy.stats.weightedtau` — atribui pesos maiores às inversões de ordem que ocorrem no topo do ranking (melhores pipelines), refletindo a importância prática de recomendar corretamente as melhores configurações primeiro.

A Figura 11 apresenta, no painel esquerdo, o histograma da distribuição de τ_w por conjunto de dados (fold LOO), e no painel direito, o diagrama de dispersão entre τ e τ_w por fold.

Figura 11 – Distribuição do Kendall Tau ponderado por conjunto de dados e comparação com τ padrão



Fonte: Autoria própria (2026).

A análise revela um padrão consistente em todos os valores de k: $\tau_w > \tau$ em média (diferença de aproximadamente 4,5 pontos percentuais para $k = 10$: $\tau_w = 0{,}522$ vs. $\tau = 0{,}477$). Este resultado indica que o preditor k-NN acerta melhor a ordenação dos pipelines de *maior desempenho* do que a dos pipelines de desempenho intermediário ou baixo. Do ponto de vista prático de AutoML, este é o comportamento mais desejável: o usuário inicia pelos primeiros pipelines recomendados, e erros nessa posição teriam o maior impacto negativo.

No diagrama de dispersão (painel direito da Figura 11), os pontos acima da diagonal confirmam que $\tau_w > \tau$ na maioria dos folds. Exceções (pontos abaixo da diagonal) representam conjuntos onde o preditor acerta a ordem global mas comete mais erros nas posições superiores — situação potencialmente problemática para uso prático.

4.5 Correlação Estatística: Embeddings e Qualidade de Ranqueamento

Uma questão central para a interpretabilidade do sistema é: *quais características dos conjuntos de dados (capturadas pelo embedding) estão associadas a uma melhor qualidade de ranqueamento pelo meta-aprendizado*?

Para investigar esta questão, calculou-se o coeficiente de Spearman entre os valores de τ_w por conjunto de dados (obtidos no LOO com $k = 10$) e cada uma das 64 dimensões do embedding, além da norma L2 do embedding de cada conjunto. A Tabela 6 apresenta as cinco dimensões com maior correlação absoluta.

Tabela 6 – Top-5 dimensões do embedding correlacionadas com o Kendall Tau ponderado por conjunto de dados ($n = 198$, LOO $k = 10$)

Dimensão	Spearman (ρ)	p-valor	Interpretação (bloco)
32	$+0{,}3633$	$< 0{,}0001$	Distribuição de percentis superiores
34	$+0{,}3469$	$< 0{,}0001$	Estatísticas de skewness por atributo
8	$-0{,}3449$	$< 0{,}0001$	Distribuição de mínimos por atributo
17	$-0{,}3378$	$< 0{,}0001$	Estatísticas de kurtosis por atributo
2	$-0{,}3350$	$< 0{,}0001$	Distribuição de médias por atributo

Fonte: Autoria própria (2026).

Todas as cinco dimensões apresentam correlações estatisticamente significativas com $p < 0{,}0001$, rejeição robusta da hipótese nula mesmo após correção conservadora de múltiplos testes. Os resultados sugerem que:

- Conjuntos com distribuições de atributos mais concentradas e menos assimétricas (correlação negativa nas dimensões de kurtosis e mínimos) tendem a ser mais fáceis de ranquear via meta-aprendizado por similaridade;
- Conjuntos com maior diversidade nos percentis superiores de seus atributos (dimensão 32, correlação positiva) também apresentam melhor qualidade de ranqueamento, possivelmente por serem mais distinguíveis no espaço de embeddings.

A análise completa, salva em `resultados/metricas/correlacao_kendall_por_dataset.csv`, constitui um passo inicial de interpretabilidade que pode orientar a seleção ou engenharia de futuros embeddings mais discriminativos para meta-aprendizado.

4.6 Demonstração Ilustrativa: Recomendação para Conjunto de Dados Retido

Para ilustrar o sistema em operação, realizou-se uma demonstração com o último conjunto do meta-conjunto (`task_id = 363713`) como holdout: o preditor foi treinado nos 200 conjuntos restantes ($k = 10$) e avaliado sobre o conjunto excluído. Esta análise tem caráter exclusivamente ilustrativo e não substitui a validação estatística principal, que é dada pelos 198 folds LOO válidos reportados anteriormente.

A Tabela 7 apresenta as métricas de ranking obtidas neste caso.

Tabela 7 – Métricas de ranking para a demonstração ilustrativa com conjunto de dados retido (`task_id = 363713`, $k = 10$)

Métrica	Valor	p-valor
Spearman (ρ)	0,9648	$< 0{,}0001$
Kendall (τ)	0,8680	$< 0{,}0001$
Kendall ponderado (τ_w)	0,8136	N/D (por construção)
NDCG	0,9995	—

Fonte: Autoria própria (2026).

O desempenho excepcional neste conjunto específico (Spearman $= 0{,}965$, NDCG $= 0{,}9995$) indica que o conjunto possui vizinhos muito próximos no espaço de embeddings — cenário ideal para o meta-aprendizado por similaridade. Por se tratar de um único caso, esses valores não são utilizados como evidência estatística independente; eles apenas exemplificam como o recomendador opera quando o conjunto de entrada encontra vizinhos informativos no meta-conjunto.

4.7 Discussão dos Resultados

Esta seção discute os resultados apresentados nas seções anteriores à luz de baselines de comparação, de resultados reportados na literatura e das limitações identificadas, interpretando o que esses achados revelam sobre a viabilidade do meta-aprendizado por similaridade para a recomendação de pipelines.

4.7.1 Comparação com Baselines Triviais

Para contextualizar os resultados e verificar se o meta-aprendizado por similaridade agrega valor além de heurísticas simples, três baselines foram estabelecidos:

1. **Ranking global médio (baseline forte):** para cada fold LOO, o ranking predito é simplesmente a ordenação dos pipelines pela sua acurácia balanceada média sobre os 200 conjuntos de treino. Este baseline captura a hipótese de que alguns pipelines são universalmente superiores — portanto não requer informação do conjunto de teste;
2. **Ranking aleatório (lower bound):** os 60 pipelines são ordenados aleatoriamente. Espera-se Spearman ≈ 0 e Kendall ≈ 0;
3. **k-NN com $k = n - 1$ (média global via vizinhança):** caso degenerado do preditor onde todos os 200 conjuntos de treino são utilizados como vizinhos, equivalendo ao baseline de ranking global médio via k-NN.

A Tabela 8 apresenta os resultados para $k = 10$ (preditor proposto) em relação aos baselines, calculados sobre os 198 folds LOO válidos.

Tabela 8 – Comparação do preditor k-NN ($k = 10$) com baselines triviais (LOO, $n = 198$ folds)

Método	Spearman (ρ)	Kendall (τ)	Kendall pond. (τ_w)
Ranking aleatório	$0{,}00 \pm 0{,}17$	$0{,}00 \pm 0{,}13$	$0{,}00 \pm 0{,}14$
Ranking global médio	$0{,}47 \pm 0{,}38$	$0{,}38 \pm 0{,}36$	$0{,}42 \pm 0{,}37$
k-NN ($k = 10$)	$\mathbf{0{,}58 \pm 0{,}36}$	$\mathbf{0{,}48 \pm 0{,}34}$	$\mathbf{0{,}52 \pm 0{,}35}$

Fonte: Autoria própria (2026).

O preditor k-NN supera o ranking global médio em todos os indicadores: $+11$ pontos percentuais em Spearman, $+10$ em Kendall e $+10$ em Kendall ponderado. Este ganho demonstra que a personalização por similaridade de conjuntos de dados contribui substantivamente além da heurística de recomendar os mesmos pipelines para todos. A diferença é particularmente relevante para o Kendall ponderado, confirmando que o meta-aprendizado melhora especialmente a recomendação das melhores configurações.

4.7.2 Comparação com Baselines e Literatura

A avaliação de sistemas de recomendação de pipelines na literatura de meta-aprendizado geralmente utiliza a correlação de Spearman como métrica principal. Os resultados desta pesquisa — Spearman médio de $0{,}58$ com LOO sobre 201 conjuntos de dados — posicionam-se de forma competitiva em relação ao estado da arte:

- Schoenfeld, Seppi e Giraud-Carrier (2022) reportam correlações de Spearman na faixa de $0{,}4$–$0{,}7$ para estimativa de desempenho de pipelines em conjuntos D3M com metamodelos baseados em estrutura de DAG, confirmando que $\rho \approx 0{,}58$ é um resultado sólido para ranqueamento via similaridade;
- Jomaa, Schmidt-Thieme e Grabocka (2021) demonstram que o Dataset2Vec supera MF2 em otimização de hiperparâmetros, com ganho típico de 5–15% em correlação de Spearman. Como o embedding deste trabalho é estatisticamente análogo ao MF2, espera-se que a substituição pelo Dataset2Vec possa melhorar os resultados em trabalhos futuros.

A Tabela 9 posiciona o presente trabalho em relação a referências selecionadas da literatura.

Tabela 9 – Posicionamento dos resultados em relação à literatura de meta-aprendizado

Trabalho	Embedding	Conjuntos	Tarefa	Spearman (LOO)
Dataset2Vec (JOMAA; SCHMIDT-THIEME; GRABOCKA, 2021)	Neural (DeepSet, 64D)	120 UCI	Otim. de hiperparâmetros	$\sim$0,55–0,70
MF2 (FEURER; SPRINGENBERG; HUTTER, 2015)	Estatístico (46D)	120 UCI	Otim. de hiperparâmetros	$\sim$0,40–0,55
Schoenfeld et al. (SCHOENFELD; SEPPI; GIRAUD-CARRIER, 2022)	DAG + metadados	194 D3M	Estimativa de desempenho	$\sim$0,40–0,70
Este TCC	**Estat. distrib. (64D)**	**201 OpenML**	**Ranqueamento de pipelines**	$\mathbf{0{,}58 \pm 0{,}36}$

Fonte: Autoria própria (2026).

Nota: os valores da literatura para Spearman foram extraídos ou estimados a partir das figuras e tabelas dos respectivos artigos; tarefas diferentes (otimização de hiperparâmetros vs. ranqueamento de pipelines) implicam que a comparação é indicativa, não diretamente equivalente.

4.7.3 Interpretação da Aparente Contradição entre NDCG e Spearman

Uma observação que merece explicação é a coexistência de NDCG médio elevado ($0{,}984 \pm 0{,}038$) com Spearman moderado ($0{,}58 \pm 0{,}36$). À primeira vista, valores de NDCG próximos a 1 sugerem que o ranking predito é quase perfeito, o que pareceria inconsistente com um Spearman de 0,58. A interpretação adequada é tratar NDCG e Kendall ponderado como métricas complementares de topo, enquanto Spearman e Kendall padrão descrevem melhor a ordenação global.

A explicação reside na diferença de sensibilidade das duas métricas. O NDCG é fortemente dominado pelas posições do topo: a contribuição da posição i decai como $1/\log_2(i+1)$, de modo que erros nas posições 40 a 60 têm peso negligenciável. Como o conjunto de pipelines avaliados com hiperparâmetros padrão frequentemente possui um subconjunto estável de "melhores pipelines"(especialmente Random Forest sem seleção de atributos, que é consistentemente superior na média), o NDCG reflete que os três a cinco primeiros pipelines recomendados são geralmente corretos.

O Spearman, por outro lado, trata todas as 60 posições com peso igual: uma inversão entre os pipelines de posição 30 e 35 penaliza tão fortemente quanto uma inversão entre os de posição 1 e 2. Como a ordenação das posições intermediárias e inferiores é genuinamente ruidosa (pipelines com acurácias próximas têm ordenações instáveis entre conjuntos de dados), o Spearman médio de 0,58 captura esta dificuldade real.

A conclusão prática é coerente: o sistema é altamente confiável para identificar os melhores pipelines (NDCG ≈ 1) e moderadamente confiável para ordenar toda a lista (Spearman $= 0{,}58$) — exatamente o comportamento desejado em aplicações de AutoML, onde o usuário raramente precisa da lista completa e apenas avalia as primeiras recomendações.

4.7.4 Embedding distribucional vs. Dataset2Vec

A adoção de um embedding distribucional (em vez do Dataset2Vec neural) foi motivada por restrições de compatibilidade, mas também possibilita uma análise direta do que o embedding captura.

Em relação aos critérios D1–D4 de Jomaa, Schmidt-Thieme e Grabocka (2021):

D1 – Independência de esquema: Atendido. O embedding distribucional é calculado para qualquer conjunto tabular, independentemente do número de atributos, instâncias ou esquema.

D2 – Expressividade: Parcialmente atendido. As 55 meta-features capturadas são mais ricas que MF1/MF2, mas potencialmente menos expressivas que embeddings neurais pré-treinados em similaridade de conjuntos de dados.

D3 – Escalabilidade: Atendido. O cálculo é $O(n \cdot p)$ sem treinamento prévio.

D4 – Correlação com meta-alvos: Atendido (evidenciado pelas correlações significativas com τ_w e pelos resultados LOO com $\rho = 0{,}58$).

4.7.5 Limitações

As principais limitações deste trabalho são:

1. **Embedding não-neural:** a ausência do Dataset2Vec real (por incompatibilidade com Python 3.13+) implica que o embedding pode ser menos expressivo que abordagens neurais pré-treinadas em similaridade. Futuros experimentos com Python 3.12 e TensorFlow podem quantificar esse ganho.

2. **Variabilidade entre conjuntos de dados:** o desvio padrão das métricas LOO ($\sigma \approx 0{,}34$–$0{,}36$) é elevado, indicando que o sistema funciona muito bem para alguns conjuntos e pobremente para outros. Uma análise mais profunda dos conjuntos com baixo τ_w poderia revelar padrões de falha sistemática.

3. **Ausência de MLP como metamodelo:** o preditor MLP (PyTorch) não foi avaliado por ausência da biblioteca no ambiente, limitando a comparação entre abordagens de meta-aprendizado.

4. **Hiperparâmetros fixos dos pipelines:** os classificadores foram usados com hiperparâmetros padrão do scikit-learn, sem ajuste individual, o que subestima o potencial de desempenho de alguns pipelines.

5 CONCLUSÃO

Este trabalho propôs e avaliou uma abordagem de meta-aprendizado baseada em embeddings estatísticos de conjuntos de dados para ranqueamento e recomendação de pipelines de aprendizado de máquina. Nesta seção, sintetizam-se os principais resultados obtidos, verifica-se o atendimento dos objetivos propostos, discutem-se as contribuições e limitações do trabalho, e apontam-se direções para pesquisas futuras.

5.1 Síntese do Trabalho

O trabalho foi estruturado em duas fases metodológicas interligadas. Na **Fase 1**, um meta-conjunto foi construído pela execução sistemática de 60 pipelines de classificação sobre 201 conjuntos de dados obtidos do OpenML, resultando em uma matriz de acurácias balanceadas de dimensão 201×60. Cada conjunto foi representado por um vetor estatístico distribucional de 64 dimensões, sem treinamento neural, que captura a distribuição de meta-features por atributo de forma invariante ao número de features e instâncias. Na **Fase 2**, um preditor k-NN multi-saída foi avaliado por Leave-One-Out no nível de conjuntos de dados, utilizando Spearman e Kendall Tau para avaliar a ordenação global, e Kendall Tau ponderado (*weightedtau*) e NDCG como métricas complementares voltadas ao topo do ranking.

Os experimentos demonstraram que o preditor k-NN com $k = 10$ obteve correlação de Spearman média de $0{,}58 \pm 0{,}36$ e Kendall Tau ponderado médio de $0{,}52 \pm 0{,}35$ sobre 198 folds LOO válidos, resultados competitivos com o estado da arte em tarefas de meta-aprendizado para seleção de algoritmos. A consistente superioridade do Kendall ponderado sobre o Kendall padrão ($+$4,5 pontos percentuais para $k = 10$), em conjunto com o NDCG elevado, indica que o sistema é particularmente mais preciso no topo do ranking — o aspecto de maior relevância prática em AutoML.

5.2 Atendimento dos Objetivos

Verifica-se o atendimento de cada objetivo específico definido na seção 1.1:

1. **Avaliar, por meio da construção de um meta-conjunto, se metadados de desempenho e meta-características de conjuntos de dados carregam sinal suficiente para diferenciar boas e más combinações de pré-processamento e classificação: Atendido.** Como pré-requisito para esta avaliação, foi construído um repositório de conhecimento (meta-conjunto) contendo $201 \times 60 = 12.060$ avaliações de pipelines com seus respectivos embeddings de conjuntos de dados, configurações de pipeline e acurácias balanceadas, armazenados em formato reutilizável (`.npy` e `.pkl`). A análise desse repositório por meio da abordagem LOO demonstrou que o meta-aprendizado

por similaridade de embeddings é capaz de ranquear pipelines de forma significativamente melhor que o acaso, com correlação de Spearman média de $0{,}58$ e correlações estatísticas com $p < 0{,}0001$.

2. **Construir e avaliar um modelo preditivo capaz de estimar, a partir de metacaracterísticas dos dados, o ranking de adequação de diferentes configurações de pré-processamento e classificação: Atendido.** O preditor k-NN multi-saída foi implementado com estrutura KDTree e avaliado por LOO. A demonstração sobre um conjunto de dados retido (Spearman $= 0{,}96$, NDCG $= 0{,}9995$) ilustra o funcionamento do sistema em um caso concreto, mas a evidência principal de generalização é a validação LOO sobre os folds válidos.

Como resultado complementar, o módulo `recomendador.py` implementa uma interface de recomendação que, dado o embedding de um novo conjunto de dados, retorna uma lista ranqueada dos 60 pipelines por desempenho estimado. A integração completa em um sistema AutoML de produção permanece como direção de trabalho futuro.

5.3 Limitações e Avaliação Crítica

Após análise observou-se as seguintes limitações do trabalho:

- **Embedding não-neural:** A incompatibilidade do TensorFlow com Python 3.13+ impediu o uso do Dataset2Vec original. O embedding estatístico adotado, embora funcional e publicável, é potencialmente menos expressivo que embeddings neurais pré-treinados em similaridade de conjuntos de dados, conforme evidenciado pelos experimentos do próprio Dataset2Vec (JOMAA; SCHMIDT-THIEME; GRABOCKA, 2021). Esta limitação representa a principal oportunidade de melhoria dos resultados.

- **Ausência do metamodelo MLP:** A indisponibilidade do PyTorch no ambiente impediu a avaliação comparativa do preditor MLP (BCE loss), que poderia capturar padrões não-lineares entre embeddings e rankings de pipelines não acessíveis ao k-NN.

- **Variabilidade elevada:** O desvio padrão das métricas LOO ($\sigma \approx 0{,}34$–$0{,}36$) reflete a heterogeneidade dos conjuntos de dados e a dependência do sistema da presença de vizinhos próximos no meta-conjunto. Para conjuntos atípicos (sem análogos no meta-conjunto), as recomendações são menos confiáveis.

- **Hiperparâmetros dos pipelines:** Os classificadores foram avaliados com hiperparâmetros padrão, sem ajuste individual, o que subestima o potencial de alguns algoritmos (especialmente SVM e MLP) e pode distorcer o ranking de qualidade dos pipelines.

5.4 Trabalhos Futuros

Com base nos resultados obtidos e nas limitações identificadas, propõem-se as seguintes extensões para trabalhos futuros:

1. **Integração com Dataset2Vec:** replicar os experimentos com o embedding neural do Dataset2Vec (JOMAA; SCHMIDT-THIEME; GRABOCKA, 2021) em um ambiente Python 3.11/3.12 com TensorFlow, quantificando o ganho de desempenho em relação ao embedding estatístico e validando a hipótese de maior expressividade (critério **D2**).

2. **Avaliação do preditor MLP:** comparar o preditor k-NN com o preditor MLP (BCE loss, PyTorch) em termos de Spearman, Kendall e NDCG, investigando se a não-linearidade do MLP resulta em melhor generalização.

3. **Expansão do arsenal de pipelines:** ampliar os 60 pipelines com combinações de ajuste de hiperparâmetros (Grid Search LOO), algoritmos adicionais (XGBoost, LightGBM, Extra Trees) e operadores de engenharia de features, aumentando a cobertura do espaço de busca de AutoML.

4. **Submissão ao BRACIS 2026:** publicação dos resultados na trilha de meta-aprendizado e AutoML do *Brazilian Conference on Intelligent Systems* (BRACIS 2026), permitindo comparação com sistemas contemporâneos e feedback da comunidade científica.

5. **Análise de transferabilidade do meta-conjunto:** investigar se o meta-conjunto construído com conjuntos OpenML generaliza para outros domínios (e.g., dados de séries temporais, imagens tabeladas), avaliando a robustez do sistema fora do contexto de treinamento.

6. **Interpretabilidade das dimensões do embedding:** aprofundar a análise das dimensões correlacionadas com τ_w (identificadas neste trabalho) para entender quais propriedades dos dados tornam o ranqueamento mais ou menos preciso, potencialmente guiando o design de embeddings mais discriminativos.

5.5 Considerações Finais

Este trabalho demonstrou que representações estatísticas distribucionais de conjuntos de dados, combinadas com um preditor k-NN e validação Leave-One-Out, constituem uma abordagem viável, reprodutível e competitiva para recomendação de pipelines em AutoML. A adoção conjunta de Kendall Tau ponderado e NDCG revelou que o sistema é particularmente preciso no topo do ranking — exatamente onde o impacto prático é maior. A análise de correlação

estatística abriu um caminho promissor para a interpretabilidade do meta-aprendizado por similaridade.

Os resultados obtidos indicam que a qualidade da representação do conjunto de dados (embedding) é o fator mais crítico para o desempenho do meta-aprendizado por similaridade — mais do que a escolha do metamodelo em si. Melhorar a representação, seja por embeddings neurais mais expressivos ou por meta-features cuidadosamente projetadas, é o caminho mais promissor para elevar a qualidade das recomendações de AutoML. O presente trabalho, ao estabelecer um baseline sólido e uma infraestrutura experimental reutilizável, contribui para este objetivo de longo prazo.

REFERÊNCIAS

ALTMAN, N. S. An introduction to kernel and nearest-neighbor nonparametric regression. **The American Statistician**, v. 46, n. 3, p. 175–185, 1992.

BENTLEY, J. L. Multidimensional binary search trees used for associative searching. **Communications of the ACM**, v. 18, n. 9, p. 509–517, 1975.

BISHOP, C. M. **Pattern Recognition and Machine Learning**. New York: Springer, 2006.

BRAZDIL, P. *et al.* **Metalearning: Applications to Data Mining**. [*S.l.*]: Springer, 2008. (Cognitive Technologies).

BREIMAN, L. Random forests. **Machine Learning**, v. 45, n. 1, p. 5–32, 2001.

BRODERSEN, K. H. *et al.* The balanced accuracy and its posterior distribution. *In*: **Proceedings of the 20th International Conference on Pattern Recognition (ICPR)**. [*S.l.*: *s.n.*], 2010. p. 3121–3124.

COHN, D.; ATLAS, L.; LADNER, R. Improving generalization with active learning. **Machine Learning**, v. 15, p. 201–221, 1994.

CORTES, C.; VAPNIK, V. Support-vector networks. **Machine Learning**, v. 20, n. 3, p. 273–297, 1995.

COVER, T. M.; HART, P. E. Nearest neighbor pattern classification. **IEEE Transactions on Information Theory**, v. 13, n. 1, p. 21–27, 1967.

DOMINGOS, P. **O algoritmo mestre: como a busca pelo algoritmo de machine learning definitivo recriará nosso mundo**. [*S.l.*]: Novatec Editora, 2017.

DOUCET, P. *et al.* Bridging diversity and uncertainty in active learning with self-supervised pre-training. 2025. Disponível em: https://arxiv.org/abs/2403.03728.

DUA, D.; GRAFF, C. **UCI Machine Learning Repository**. 2019. University of California, Irvine, School of Information and Computer Sciences. Disponível em: https://archive.ics.uci.edu/ml.

EDWARDS, H.; STORKEY, A. Towards a neural statistician. *In*: **International Conference on Learning Representations (ICLR)**. [*S.l.*: *s.n.*], 2017.

FERNÁNDEZ-DELGADO, M. *et al.* Do we need hundreds of classifiers to solve real world classification problems? **Journal of Machine Learning Research**, v. 15, n. 1, p. 3133–3181, 2014.

FEURER, M. *et al.* Efficient and robust automated machine learning. *In*: **Advances in Neural Information Processing Systems 28 (NIPS 2015)**. [*S.l.*]: Curran Associates, Inc., 2015. p. 2962–2970.

FEURER, M.; SPRINGENBERG, J. T.; HUTTER, F. Initializing bayesian hyperparameter optimization via meta-learning. *In*: **Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence**. Austin, Texas, USA: AAAI Press, 2015. p. 1128–1135.

GOLSHANRAD, P. *et al.* Mega: Predicting the best classifier combination using meta-learning and a genetic algorithm. **Intelligent Data Analysis**, v. 25, n. 6, p. 1547–1563, 2021. Disponível em: https://journals.sagepub.com/doi/abs/10.3233/IDA-205494.

GUYON, I.; ELISSEEFF, A. An introduction to variable and feature selection. **Journal of Machine Learning Research**, v. 3, p. 1157–1182, 2003.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. **Neural Computation**, v. 9, n. 8, p. 1735–1780, 1997.

JOLLIFFE, I. T. **Principal Component Analysis**. 2. ed. New York: Springer, 2002.

JOMAA, H. S.; SCHMIDT-THIEME, L.; GRABOCKA, J. Dataset2vec: Learning dataset meta-features. **Data Mining and Knowledge Discovery**, v. 35, n. 3, p. 964–985, 2021.

KENDALL, M. G. A new measure of rank correlation. **Biometrika**, v. 30, n. 1/2, p. 81–93, 1938.

KHAN, I. *et al.* Autofe-sel: A meta-learning based methodology for recommending feature subset selection algorithms. **KSII Transactions on Internet and Information Systems**, v. 17, p. 1773–1793, 07 2023.

LIPPMANN, R.; CAMPBELL, W.; CAMPBELL, J. An overview of the DARPA data driven discovery of models (D3M) program. *In*: **29th Conference on Neural Information Processing Systems (NIPS)**. Barcelona, Spain: [*s.n.*], 2016.

MAHER, M.; SAKR, S. SmartML: A meta learning-based framework for automated selection and hyperparameter tuning for machine learning algorithms. *In*: **Proceedings of the 22nd International Conference on Extending Database Technology (EDBT 2019)**. Lisboa, Portugal: [*s.n.*], 2019. p. 554–557.

OLSON, R. S. *et al.* Automating biomedical data science through tree-based pipeline optimization. *In*: **Applications of Evolutionary Computation**. [*S.l.*]: Springer, 2016. p. 123–137.

PEDREGOSA, F. *et al.* Scikit-learn: Machine learning in python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011.

PENG, H. A comprehensive overview and survey of recent advances in meta-learning. **CoRR**, abs/2004.11149, 2020. Disponível em: https://arxiv.org/abs/2004.11149.

PEREIRA, G. T.; SANTOS, M. R. dos; CARVALHO, A. C. P. de Leon Ferreira de. Evaluating meta-feature selection for the algorithm recommendation problem. **CoRR**, abs/2106.03954, 2021. Disponível em: https://arxiv.org/abs/2106.03954.

QUINLAN, J. R. Induction of decision trees. **Machine Learning**, v. 1, n. 1, p. 81–106, 1986.

RAMAMURTHY, K. N.; VARSHNEY, K. R.; MODY, K. **Topological Data Analysis of Decision Boundaries with Application to Model Selection**. 2018. Disponível em: https://arxiv.org/abs/1805.09949.

REIF, M.; SHAFAIT, F.; DENGEL, A. Meta2-features: Providing meta-learners more information. *In*: **Poster and Demo Track of the 35th German Conference on Artificial Intelligence (KI)**. [*S.l.*: *s.n.*], 2012.

SCHOENFELD, B.; SEPPI, K.; GIRAUD-CARRIER, C. Metalearning using structure-rich pipeline representations for improved automl. **International Journal of Data Analysis Techniques and Strategies**, v. 14, n. 1, p. 49–72, 2022. Disponível em: https://www.inderscienceonline.com/doi/abs/10.1504/IJDATS.2022.129174.

SCHOENFELD, B. J. **Metalearning by Exploiting Granular Machine Learning Pipeline Metadata**. 2020. Dissertação (Mestrado) — Brigham Young University, 2020. Disponível em: https://scholarsarchive.byu.edu/etd/8730/.

SENER, O.; SAVARESE, S. Active learning for convolutional neural networks: A core-set approach. *In*: **International Conference on Learning Representations (ICLR)**. [*S.l.*: *s.n.*], 2018.

SETTLES, B. **Active Learning Literature Survey**. [*S.l.*], 2009. Disponível em: https://minds.wisconsin.edu/bitstream/handle/1793/60660/TR1648.pdf?sequence=1&isAllowed=y.

SPEARMAN, C. The proof and measurement of association between two things. **The American Journal of Psychology**, v. 15, n. 1, p. 72–101, 1904.

TRIANTAFILLOU, E. *et al.* Meta-dataset: A dataset of datasets for learning to learn from few examples. *In*: **International Conference on Learning Representations (ICLR)**. [*s.n.*], 2020. Disponível em: https://arxiv.org/abs/1903.03096.

VANSCHOREN, J. Meta-learning: A survey. **arXiv preprint arXiv:1810.03548**, 2018. Disponível em: https://arxiv.org/abs/1810.03548.

VANSCHOREN, J. *et al.* Openml: Networked science in machine learning. **ACM SIGKDD Explorations Newsletter**, v. 15, n. 2, p. 49–60, 2014. Disponível em: https://www.kdd.org/exploration_files/15-2-2013-12.pdf#page=51.

VARSHNEY, K. R.; RAMAMURTHY, K. N. Persistent topology of decision boundaries. **ICASSP**, IEEE, p. 3931–3935, 2015.

VILALTA, R.; DRISSI, Y. A perspective view and survey of meta-learning. **Artificial Intelligence Review**, v. 18, p. 77–95, 2002.

WANG, G.; SONG, Q.; ZHU, X. **Ensemble Learning Based Classification Algorithm Recommendation**. 2021. Disponível em: https://arxiv.org/abs/2101.05993.

WISTUBA, M.; SCHILLING, N.; SCHMIDT-THIEME, L. Two-stage transfer surrogate model for automatic hyperparameter optimization. *In*: **Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML/PKDD)**. [*S.l.*: *s.n.*], 2016. p. 13–28.

WOLPERT, D. H.; MACREADY, W. G. No free lunch theorems for optimization. **IEEE Transactions on Evolutionary Computation**, v. 1, n. 1, p. 67–82, 1997.